ed-tech collaboratory

Assessment Vendor Ed-Fi Native Integration Playbook

Part I

1 Executive Summary

1.1 Purpose

The purpose of this playbook is to define clear guidance and foundational design principles for what constitutes a native Ed-Fi assessment integration.

Ed-Fi is a data standard and API-based interoperability framework used by education agencies to exchange data consistently across domains such as assessment, enrollment, attendance, grades, and demographics. In many implementations, this standard is operationalized through the Ed-Fi ODS/API, which provides a common interface for data exchange, and the Operational Data Store (ODS), which stores and manages the data. For assessment providers, supporting Ed-Fi is not simply about delivering a file or exposing data elsewhere. It means integrating into a common, structured exchange model that enables assessment data to flow into a broader educational data ecosystem in a usable, governed way.

The Ed-Fi Assessment domain is intentionally flexible. It was designed to accommodate the wide variety of assessment instruments used across K–12 systems, including state summative assessments, formative screeners, interim benchmarks, college-readiness exams, diagnostic tools, and classroom-level measures. The model supports hierarchical structures, multiple score types, performance levels, and alignment to learning standards. That flexibility is powerful, and it is not self-governing. Without modeling discipline and governance alignment, the same assessment can be implemented in materially different ways across vendors, states, and districts. Data may technically load yet fail to support real-world analytics.

This playbook establishes a clear standard: a native Ed-Fi integration is not simply a data feed that posts results to an Ed-Fi ODS. It is an integration that publishes a complete, hierarchically accurate, subject-safe, and governance-aligned representation of an assessment, suitable for both compliance reporting and local analytic use without requiring custom downstream logic.

This playbook also makes it explicit that native integration is a full lifecycle responsibility, not just a data modeling exercise. It covers the major implementation workstreams required to move from raw vendor data to a production-ready integration, including data modeling, student identity and rostering, descriptor governance, integration architecture, operational readiness, and testing and certification.

1.2 What Districts and States Need from Assessment Data

Districts and states do not use assessment data as isolated scores. They rely on it to support operational, instructional, and policy decisions that require complete, structured, and longitudinally consistent data.

In practice, stakeholders expect to answer questions such as:

- How did this student grow from the beginning of the year to the end of the year?
- How are outcomes trending by subgroup, school, or program participation?
- How do students perform across different vendors' assessments in a single dashboard?
- What was the student's performance at the time they were enrolled in a specific school?
- How do subdomain or strand-level results inform instruction and intervention?
- How do assessment outcomes connect to college and career readiness indicators?

These are not advanced use cases. They are baseline expectations for districts, state agencies, and research partners.

However, most assessment integrations are designed to satisfy a much narrower requirement: state reporting. A typical reporting specification may require only a composite score, a performance level, and a testing window or period indicator. While sufficient for compliance, this level of data is not sufficient for real-world use.

As a result, critical elements of the assessment are often lost or inconsistently represented. Subscores, growth measures, and contextual indicators may be omitted. Hierarchical structure may be flattened or partially modeled. Event context, such as the administration date or grade level, may be incomplete. Vendor-specific meaning may be lost or inconsistently interpreted.

The impact is predictable. Districts and states must reconstruct missing structures, reverse-engineer meaning, maintain vendor-specific transformation logic, and reconcile inconsistencies across systems and over time. This increases cost, delays access to insights, and limits the usability of the data.

A native Ed-Fi assessment integration is designed to eliminate these gaps.

It ensures that assessment data supports state reporting, district and school-level dashboards, longitudinal growth analysis, subgroup and program evaluation, and cross-vendor comparability—without requiring additional downstream engineering.

Completeness at ingestion enables usability downstream. When the full structure, context, and meaning of an assessment are preserved at the point of integration, the data becomes immediately usable across systems, partners, and use cases.

1.3 The Problem This Playbook Addresses

Despite widespread adoption of Ed-Fi, assessment integrations remain inconsistent, fragile, and difficult to scale. Limitations in the standard itself do not cause these challenges; rather, they stem from variability in how integrations are implemented across vendors and environments.

Four core problem areas consistently emerge:

Data Modeling Inconsistency

Vendors make different decisions about how to represent the same assessment in Ed-Fi, including how to define assessment identity, structure the hierarchy, assign subjects, and place results at the correct level in the student assessment record. These variations often produce datasets that pass API validation but cannot be analyzed consistently across districts, states, or vendors.

Without a shared modeling approach, analytics must be rewritten for each integration, undermining interoperability and increasing implementation cost.

Student Identity Mismatches

Assessment results must resolve to a valid StudentUniqueId to be usable. In practice, mismatches between vendor identifiers and district or state identity systems lead to rejected records, incomplete datasets, or silent data gaps.

Even low rates of identity misalignment can undermine longitudinal analysis, distort reporting, and require manual reconciliation efforts that do not scale.

Descriptor Governance Failures

Descriptors define the meaning of data elements. When vendor-specific values are overwritten, reused incorrectly, or inconsistently mapped, the data's semantic integrity is lost.

This results in ambiguity between scores and performance levels, inconsistency across domains such as assessment and courses, and a lack of transparency in how values are interpreted downstream.

Operational Reliability Gaps

Many integrations are not designed for repeatable, large-scale operation. Missing dependency enforcement, weak reprocessing logic, inconsistent scheduling, and limited monitoring create systems that are difficult to maintain and prone to failure.

These gaps lead to incomplete loads, duplicate records, delayed data availability, and a high ongoing support burden for both vendors and education agencies.

The result across all four areas is the same: technically valid data that cannot be reliably used without vendor-specific transformation, manual intervention, or parallel data pipelines.

This playbook addresses these problems by defining a consistent, end-to-end standard for how assessment integrations should be modeled, implemented, and operated.

1.4 What This Playbook Covers

This playbook defines the full lifecycle of a native Ed-Fi assessment integration across six core workstreams. Each workstream represents a critical component of a complete, scalable, and analytically usable integration, and together they establish the expectations for moving from raw assessment data to a production-ready implementation.

Data Modeling defines how assessments, hierarchical structure, results, their placement within the student assessment record, and descriptors must be represented in the Ed-Fi data model. It establishes the rules that ensure assessment data is complete, interpretable, and consistent across vendors and environments.

Student Identity and Rostering establishes how assessment results are aligned to StudentUniqueId and connected to the broader data ecosystem. It defines how identity is resolved, validated, and maintained, enabling results to support longitudinal analysis and enrollment-aligned reporting.

Descriptor Governance defines how meaning is preserved at ingestion through proper use of namespaces and descriptor categories. It also establishes how mapping and interpretation are handled transparently, ensuring data remains consistent across domains and implementations.

Integration Architecture specifies how data is transformed, validated, and transmitted into the Ed-Fi ODS/API. It defines the technical patterns that ensure integrations are reliable, repeatable, and scalable across multiple partners.

Operational Readiness defines the runtime expectations for operating an integration in production. This includes scheduling, retry behavior, monitoring, error handling, and change management to ensure ongoing stability and maintainability.

Testing and Certification establishes the validation criteria for confirming that the integration meets the requirements of this playbook. It ensures that integrations are not only technically correct but also complete, stable, and analytically usable.

Together, these workstreams define what it takes to move from a data feed that loads to an integration that can be trusted, reused, and scaled across districts, states, and vendors.

1.5 The Business Case for Vendors

Adopting a native Ed-Fi assessment integration is not just a technical decision. It is a business decision that directly affects costs, scalability, and the long-term product strategy.

Today, most assessment integrations are built and maintained as one-off implementations. Variability across states, districts, and partners—particularly in areas such as rostering, student identity, descriptor usage, and data expectations—forces vendors into a repeated cycle of customization, support, and rework.

This playbook defines a different model: build once, implement consistently, and scale across partners.

Without a standardized approach, vendors incur ongoing operational and engineering costs:

Per-site customization cycles
Each implementation requires adapting to different rostering formats, identifier conventions, and data expectations. Inconsistent rostering approaches across implementations create significant overhead in aligning student identity and preparing data for integration.

Sustained support burden
Fragmented integrations lead to recurring issues with identity mismatches, missing data, and inconsistent behavior across environments. Support teams must continuously troubleshoot problems that stem from a lack of standardization rather than product defects.

Rework across implementations and certifications
Integrations built for one state or partner often require redesign when deployed elsewhere. As expectations evolve, especially with emerging standards, vendors are forced to revisit and retrofit prior work.

With a native integration aligned to this playbook, vendors establish a scalable and repeatable foundation for growth:

Scalable, repeatable deployment model
A consistent integration pattern reduces the need for per-partner customization, enabling faster onboarding across states and districts.

Clear alignment with procurement expectations
As ETC states, adopt this playbook within RFPs and vendor evaluation processes, and alignment to these standards becomes a differentiator and a pathway to broader adoption.

Future alignment with Ed-Fi Data Standard direction
Implementing these principles positions vendors for compatibility with Data Standard 6.0 and reduces the need for future rework as the ecosystem evolves.

Participation in a stable, interoperable ecosystem
Standardized integrations reduce fragmentation and enable vendors to operate in a more predictable, consistent data exchange environment.

A native integration requires upfront investment and a shift from one-off delivery to standardized implementation:

Initial investment in modeling and architecture
Vendors must design integrations that fully reflect the assessment structure, including how results are structured within student assessment records, reliably resolve student identity, and implement governance of descriptor handling and operational safeguards.

Commitment to consistency across implementations
The value of this approach lies in applying the same integration patterns across partners rather than adapting the model for each new environment.

Ongoing operational discipline
Integrations must be maintained with clear versioning, monitoring, and governance practices to ensure stability as both vendor products and Ed-Fi standards evolve.

A native Ed-Fi integration shifts that model by establishing a consistent, repeatable approach that reduces long-term cost, improves reliability, and aligns with emerging expectations across states and the broader ecosystem. The investment is upfront. The return is sustained.

1.6 Foundational Design Principles

The following principles define the governing logic of a native Ed-Fi assessment integration. They are not implementation details. They are the constraints that ensure assessment data remains complete, interpretable, and reusable across systems and over time.

These principles apply across all workstreams in this playbook and should guide every modeling, architecture, and operational decision that follows.

Send the Full Score Report
If a data element appears on the official score report and is available in vendor exports, it belongs in Ed-Fi unless a regulatory constraint prohibits it. This includes composite scores, subscores, percentiles, growth metrics, performance levels, risk indicators, contextual flags, accommodations, and administration conditions.

Publishing only the minimum required for state reporting results in incomplete datasets that cannot support real-world use. Ed-Fi is not a reporting file destination. It is an interoperability standard.

Model the Assessment Hierarchy

Assessment data must reflect the structure of the vendor's scorebook. The Ed-Fi model provides explicit constructs to represent this hierarchy, including Assessment, ObjectiveAssessment, and StudentAssessment, which contains StudentObjectiveAssessment as a collection of objective-level results.

Top-level results belong at the student assessment level. Subscores belong within the StudentObjectiveAssessment collection inside the StudentAssessment record. If the vendor presents a hierarchical score report, the integration must preserve that structure. Collapsing hierarchy for convenience compromises analytical integrity.

Protect Subject Integrity (DS 6.0 Alignment)

Each assessment must resolve to exactly one academic subject. When an assessment produces cross-subject results, the top-level subject must be represented as *Composite*, with subject-specific results modeled within the hierarchy.

Allowing multiple subjects at the top level forces downstream systems to infer meaning, which is not stable or reliable. Subject meaning must be explicit at ingestion.

Separate Scores from Performance Levels

Scores are quantitative measures. Performance levels are categorical interpretations. These concepts are distinct and must be modeled separately.

Treating performance levels as scores or assuming a one-to-one relationship between them reduces analytical clarity and introduces ambiguity into downstream use.

Preserve Vendor Semantics

Assessment-specific descriptors, including score names and performance levels, must retain vendor-defined meaning within the vendor namespace. Normalization to shared analytic categories should occur downstream, where context is known.

Preserving vendor semantics maintains transparency, avoids hidden transformation logic, and allows multiple interpretations of the same data without loss of meaning.

Include Full Event Context

Assessment results must include the contextual information required to interpret them correctly. This includes the administration date, school year, assessment period, grade level at the time of assessment, and relevant attempt indicators when available.

Without full event context, results cannot be reliably aligned to enrollment, compared across time, or used for longitudinal analysis.

Align to Governance Oversight

Assessment modeling carries inherent ambiguity, including how identifiers are defined, how descriptors are managed, and how changes are versioned over time.

A native integration must align with a governance framework that defines identifier strategy, namespace ownership, descriptor management, versioning rules, and change control expectations. Without governance alignment, consistency degrades over time, even if the initial implementation is correct.

These principles establish the expectations for how assessment data must be represented, interpreted, and maintained. The sections that follow translate these principles into concrete modeling, identity, architecture, and validation requirements.

1.7 The Standard

A native Ed-Fi assessment integration is one that publishes a complete, hierarchically accurate, subject-safe, and governance-aligned representation of an assessment that enables stakeholders to analyze student performance reliably, measure growth over time, and compare outcomes across systems without requiring custom downstream reconstruction, score parsing, or vendor-specific transformation logic.

An integration that merely loads data is not sufficient. An integration must interoperate.

Part II – How Ed-Fi Assessment Data Works

2. The Ed-Fi Ecosystem

2.1 What the ODS is and How the API Works

Ed-Fi provides a standardized way for education systems and technology providers to exchange data across domains, including assessment, enrollment, attendance, grades, and demographics. This is accomplished through two core components: the Operational Data Store (ODS) and the RESTful API.

The ***Operational Data Store (ODS)*** is where data is stored. It is a structured database that organizes information according to the Ed-Fi data model, enabling data from different sources to be represented consistently. Assessment data does not exist in isolation within the ODS. It is connected to other domains such as students, schools, courses, and programs, enabling a more complete and integrated view of student outcomes.

The ***API*** is how data enters and exits the system. It provides a standardized interface that allows external systems, including assessment platforms, to submit and retrieve data using common patterns. Rather than exchanging custom files or managing one-off data extracts, vendors interact with a consistent set of endpoints that represent Ed-Fi resources such as Assessment, StudentAssessment, and its associated structures.

These two components work together as a single system. The API serves as the access layer, handling validation, authorization, and data exchange, while the ODS serves as the storage layer, maintaining the structured data that supports downstream reporting and analytics. Vendors do not interact directly with the database. All data exchange occurs through the API.

Access to the API is controlled by implementation-specific credentials issued for each environment. These credentials determine what data can be submitted and accessed. Permissions are managed through predefined access configurations, often called claimsets, that define the scope of allowed interactions with the API.

In practice, this means that an assessment vendor integrates into a defined environment by authenticating with the API and submitting data in alignment with the Ed-Fi data model. The system receiving the data may represent a district, a state, or another education agency implementation, but the interaction model remains consistent.

The key concept is straightforward: the API is how data is exchanged, and the ODS is where that data is stored and connected to the broader ecosystem. Understanding this relationship is essential before deciding how assessment data should be modeled, transmitted, and validated within an Ed-Fi integration.

2.2 The Assessment Domain: Core Entities

The Ed-Fi Assessment domain is designed to represent how assessment results are interpreted and used in real-world education settings. It is not intended to replicate the full complexity of an assessment platform, such as item banks, delivery engines, or scoring algorithms. Instead, it provides a structured way to describe assessment results so they can be consistently exchanged, understood, and used across systems.

At its core, the domain is built around a set of core entities that work together to represent both the structure of an assessment and the results produced for each student. These constructs operate across two parallel layers:

- *Assessment and ObjectiveAssessment* entities define what the assessment is and how it is structured.
- *StudentAssessment* entity and its corresponding *StudentObjectiveAssessment collection* record what happened for a student within that structure.

The Core Entities

Assessment
The Assessment entity represents the assessment instrument at the highest level. It defines the overall context in which results are reported, including the intended subject, reporting structure, and the hierarchy of components that make up the assessment.

ObjectiveAssessment
ObjectiveAssessment represents the subcomponents of an assessment, such as sections, strands, domains, measures, or skill groupings. Many assessments organize results into multiple levels, and this entity allows that structure to be represented.

Some assessments include detailed subscores and multi-level hierarchies, while others report only a single overall result. The model supports both cases by allowing hierarchy where it exists and remaining simple where it does not.

StudentAssessment
StudentAssessment represents a student's attempt at an assessment. It captures the overall outcome of that attempt along with the contextual information needed to interpret it, such as when the assessment occurred. It reflects the highest level of results for a student.

> *StudentObjectiveAssessment*
> StudentObjectiveAssessment is a collection within the *StudentAssessment* record that represents a student's results within the assessment structure, such as performance at the strand, domain, or skill level. These results align with the corresponding ObjectiveAssessment definitions and provide the details needed for deeper analysis.

A useful way to think about this model is:

- Assessment describes the *map*
- ObjectiveAssessment defines the *landmarks within the map*
- StudentAssessment records the *journey,* and the collection of StudentObjectiveAssessment records what happened at each *landmark*

This separation preserves meaning while ensuring results remain consistent, interpretable, and comparable across assessments and vendors.

Optional Entities

In some cases, additional detail may be included when supported by the assessment and required by the implementation context.

AssessmentItem (optional)
Represents item-level results. This level of detail can support advanced analysis but introduces additional complexity and is typically used only when item-level reporting is required and governed.

LearningStandard (optional)
Represents the alignment between assessment components and academic standards. This can support standards-based analysis but requires careful management because standards vary across states and may change over time.

The Ed-Fi Assessment domain provides a flexible structure that can represent a wide range of assessment types while maintaining a consistent model for interpretation. Understanding how these core entities and their nested result structures relate is essential before deciding how to model, integrate, and use assessment data.

2.3 Descriptors and Namespaces

Two foundational concepts when working with Ed-Fi are *descriptors* and *namespaces*. These define both the representation of categorical meaning and its ownership, which are critical for preserving semantic integrity across integrations.

A ***descriptor*** represents a controlled vocabulary value. It is how Ed-Fi expresses categorical meaning in a consistent, machine-readable way. Examples include score types, performance levels, and assessment periods. Descriptors are not just labels. They are governed values that must be interpreted consistently across systems.

A ***namespace*** represents *ownership of meaning*. It defines who is responsible for the definition and lifecycle of a descriptor. This is a critical distinction in Ed-Fi:

- The default Ed-Fi namespace is used for shared, cross-domain concepts (such as AcademicSubject or GradeLevel).
- A vendor namespace is used for assessment-specific semantics that originate from the provider (such as score names or performance levels).

Ed-Fi separates these namespaces intentionally. This separation ensures that:

- Vendor meaning is preserved exactly as reported
- Shared concepts remain consistent across domains
- Downstream systems can interpret or map values without losing traceability

For example, consider a vendor that reports a score called "*Lexile Measure*."

API Integration
The score is sent using an AssessmentReportingMethodDescriptor in the vendor namespace, for example:
uri://vendor.org/AssessmentReportingMethodDescriptor#LexileMeasure

This preserves the exact meaning defined by the vendor.

Within the Ed-Fi ODS
The value remains unchanged. No normalization or renaming occurs at this stage. The system stores the vendor's report.

This pattern is essential. It ensures that:

- The original vendor's meaning is never lost
- Local interpretations are explicit and auditable
- Multiple vendors can coexist without forcing premature standardization

2.4 Student Identity in Ed-Fi

A foundational concept in Ed-Fi is the use of the *StudentUniqueId* as the canonical anchor for student identity. All student-level data, including assessment results, must resolve to the student represented by this identifier in order to be successfully ingested, linked, and reported.

This requirement is critical because Ed-Fi does not rely on names or local identifiers for matching. Instead, the StudentUniqueId ensures that records are consistently associated with the correct student across systems, time, and data domains.

When assessment data does not correctly resolve to the student represented by a valid StudentUniqueId, several issues can occur:

- Payloads may be rejected during ingestion
- Records may fail to associate with a student, creating silent data gaps
- Longitudinal reporting may be incomplete or inaccurate

These failure points are often not immediately visible but can significantly impact downstream reporting and analytics.

This section introduces the concept at a high level. A more detailed treatment of identity resolution, matching strategies, and common implementation challenges is provided in Part IV.

2.5 Worked Example: A Benchmark Assessment End to End

This worked example introduces a hypothetical single-subject benchmark assessment — the **ClearPath K–3 Reading Benchmark** — and walks through how it is completely represented in Ed-Fi. It covers the Assessment definition, the three ObjectiveAssessment records that define its subscore hierarchy, a StudentAssessment event record, and the StudentObjectiveAssessment results collection nested within it. Descriptor usage, namespace ownership, and event context fields are shown in concrete annotated payloads. This example serves as the primary reference point throughout the remainder of the Playbook.

Assessment overview

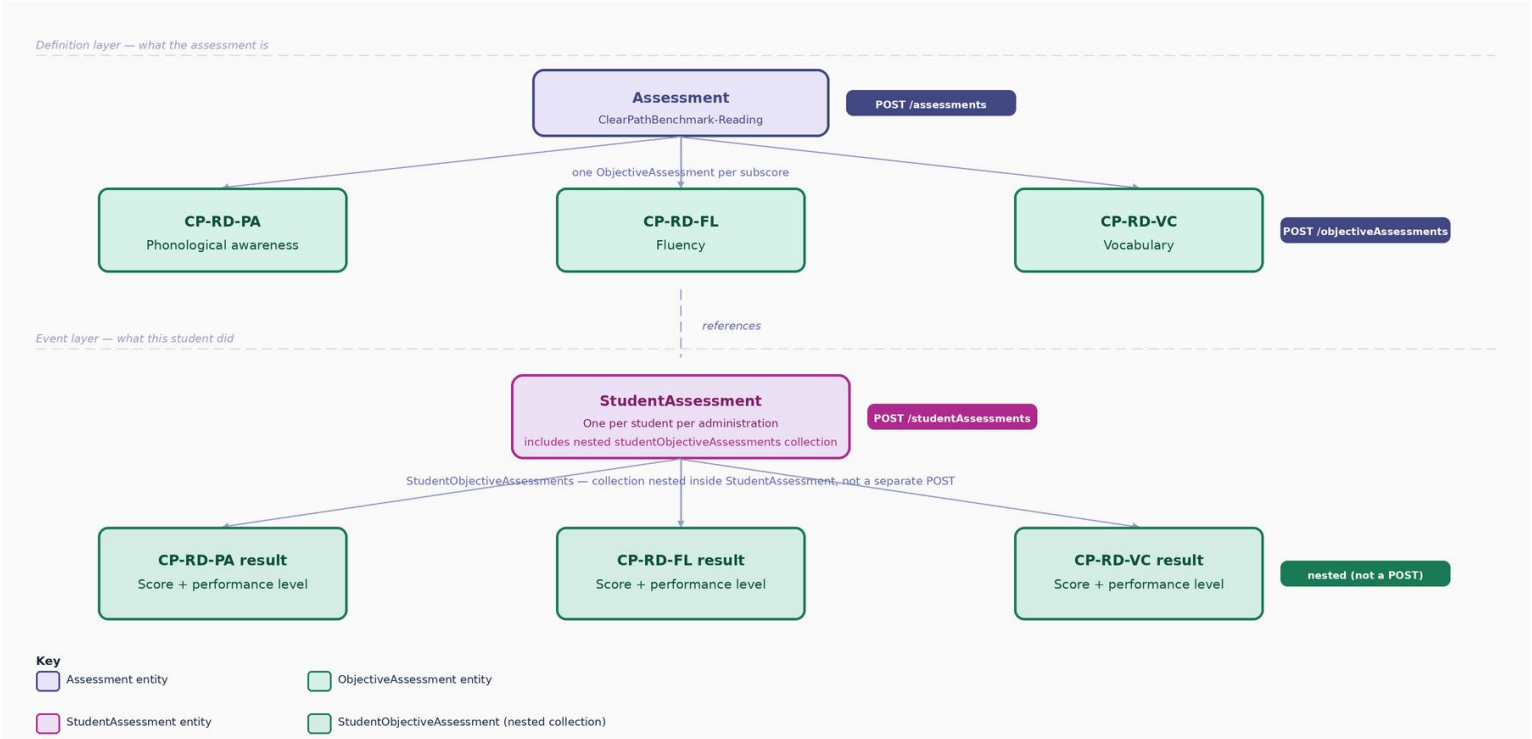
ClearPath is a hypothetical assessment vendor offering a benchmark screener for grades K–3. The Reading Benchmark yields a composite score plus three subscore areas: Phonological Awareness, Fluency, and Vocabulary. The assessment is administered three times per year (fall, winter, spring). This example covers one student's fall administration.

Attribute	Value
Assessment name	ClearPath K–3 Reading Benchmark
Subject	English Language Arts (one subject — one Assessment record)
Assessed grades	Kindergarten, Grade 1, Grade 2, Grade 3
Administration periods	Fall, Winter, Spring
Vendor namespace	uri://clearpath.example.com/assessment/clearpath-reading
Assessment identifier	ClearPathBenchmark-Reading
Subscores	CP-RD-PA (Phonological Awareness), CP-RD-FL (Fluency), CP-RD-VC (Vocabulary)

> **GOVERNANCE — SECTION 4: ONE SUBJECT PER ASSESSMENT**
>
> This example intentionally covers one subject only. If ClearPath also offers a Math screener, that is a separate Assessment record with its own assessmentIdentifier, its own ObjectiveAssessments, and its own StudentAssessment events. A single Assessment record must never span two academic subjects. The two assessments share a vendor and a namespace but are otherwise independent entities in Ed-Fi. Placing both Reading and Math results under one Assessment identifier collapses the subject boundary, prevents subject-level analytics, and violates the single-subject rule defined in Section 4.

Entity hierarchy

The Ed-Fi assessment domain uses two parallel layers. The definition layer describes the assessment structure. The event layer records what a specific student did on a specific date. One Assessment record anchors three ObjectiveAssessments. One StudentAssessment event carries three nested StudentObjectiveAssessment results — one per subscore. The relationships are shown in the table below.



Ed-Fi Assessment Entity Hierarchy — ClearPath K-3 Reading Benchmark

Section 2.5 · ETC Assessment Integration Playbook

Layer	Entity	Role
Definition	**Assessment**	Defines the map: one record for the Reading Benchmark
Definition	**ObjectiveAssessment**	Defines the landmarks: one record per subscore (CP-RD-PA, CP-RD-FL, CP-RD-VC)
Event	**StudentAssessment**	Records the journey: one record per student per administration
Event	**StudentObjectiveAssessment**	Records results at each landmark: a collection nested inside StudentAssessment, not a separate POST

PLAYBOOK REFERENCE — SECTION 2.2: PARALLEL LAYERS

StudentObjectiveAssessment is not a standalone entity posted via a separate API call. It is a collection nested within the StudentAssessment body. The definition layer (Assessment + ObjectiveAssessments) describes the map. The event layer (StudentAssessment + the nested collection) records the student's journey through that map.

Step 1 — POST /ed-fi/assessments

The Assessment resource defines the map. One Assessment record is posted for the Reading Benchmark. ClearPath owns the namespace, the assessmentIdentifier, and all vendor-specific descriptors. These values must remain stable across all school years and all states where this assessment is deployed.

[POST] /ed-fi/assessments


```
{
  "assessmentIdentifier": "ClearPathBenchmark-Reading",
  "namespace": "uri://clearpath.example.com/assessment/clearpath-reading",
  "assessmentTitle": "ClearPath K-3 Reading Benchmark",
  "assessmentFamily": "ClearPath Benchmark",
  "assessmentVersion": 4,
  "revisionDate": "2023-07-01",
  "assessedGradeLevels": [
   { "gradeLevelDescriptor": "uri://ed-fi.org/GradeLevelDescriptor#Kindergarten" },
   { "gradeLevelDescriptor": "uri://ed-fi.org/GradeLevelDescriptor#First grade" },
   { "gradeLevelDescriptor": "uri://ed-fi.org/GradeLevelDescriptor#Second grade" },
   { "gradeLevelDescriptor": "uri://ed-fi.org/GradeLevelDescriptor#Third grade" }
  ],
  "academicSubjects": [
   { "academicSubjectDescriptor": "uri://ed-fi.org/AcademicSubjectDescriptor#English Language Arts" }
  ],
  "scores": [
   {
    "assessmentReportingMethodDescriptor":
     "uri://clearpath.example.com/assessment/clearpath-
reading/AssessmentReportingMethodDescriptor#Composite Score",
    "resultDatatypeTypeDescriptor": "uri://ed-fi.org/ResultDatatypeTypeDescriptor#Integer",
    "minimumScore": "0",
    "maximumScore": "1000"
   },
   {
    "assessmentReportingMethodDescriptor":
```

```
      "uri://clearpath.example.com/assessment/clearpath-reading/AssessmentReportingMethodDescriptor#SEM",
     "resultDatatypeTypeDescriptor": "uri://ed-fi.org/ResultDatatypeTypeDescriptor#Integer"
    }
  ],
  "performanceLevels": [
   {
    "assessmentReportingMethodDescriptor":
     "uri://clearpath.example.com/assessment/clearpath-
reading/AssessmentReportingMethodDescriptor#Benchmark Level",
    "performanceLevelDescriptor":
     "uri://clearpath.example.com/assessment/clearpath-reading/PerformanceLevelDescriptor#Did not yet meet
expectations",
    "resultDatatypeTypeDescriptor": "uri://ed-fi.org/ResultDatatypeTypeDescriptor#Level"
   },
   {
    "assessmentReportingMethodDescriptor":
     "uri://clearpath.example.com/assessment/clearpath-
reading/AssessmentReportingMethodDescriptor#Benchmark Level",
    "performanceLevelDescriptor":
     "uri://clearpath.example.com/assessment/clearpath-reading/PerformanceLevelDescriptor#Approaching
expectations",
    "resultDatatypeTypeDescriptor": "uri://ed-fi.org/ResultDatatypeTypeDescriptor#Level"
   },
   {
    "assessmentReportingMethodDescriptor":
     "uri://clearpath.example.com/assessment/clearpath-
reading/AssessmentReportingMethodDescriptor#Benchmark Level",
    "performanceLevelDescriptor":
     "uri://clearpath.example.com/assessment/clearpath-reading/PerformanceLevelDescriptor#Met expectations",
    "resultDatatypeTypeDescriptor": "uri://ed-fi.org/ResultDatatypeTypeDescriptor#Level"
   },
   {
    "assessmentReportingMethodDescriptor":
```

```
      "uri://clearpath.example.com/assessment/clearpath-
reading/AssessmentReportingMethodDescriptor#Benchmark Level",

    "performanceLevelDescriptor":

     "uri://clearpath.example.com/assessment/clearpath-reading/PerformanceLevelDescriptor#Exceeded
expectations",

    "resultDatatypeTypeDescriptor": "uri://ed-fi.org/ResultDatatypeTypeDescriptor#Level"

  }

 ]

}
```

GOVERNANCE — SECTION 3: ASSESSMENT IDENTITY

The assessmentIdentifier + namespace pair is the identity contract for this assessment. Both values must remain identical across every school year and every state deployment. Changing either value breaks longitudinal tracking and creates orphaned student records that can no longer be matched to their assessment definition. The namespace must use a domain owned and controlled by ClearPath, not by any state or district.

DESCRIPTOR OWNERSHIP — SECTION 7

AssessmentReportingMethodDescriptor and PerformanceLevelDescriptor values carry the vendor namespace prefix (uri://clearpath.example.com/...). These are vendor-owned descriptors and must not be overridden, normalized, or replaced at ingestion. GradeLevelDescriptor, AcademicSubjectDescriptor, and ResultDatatypeTypeDescriptor use the Ed-Fi shared namespace (uri://ed-fi.org/...) because they are cross-domain descriptors governed by the Ed-Fi Alliance.

Step 2 — POST /ed-fi/objectiveAssessments (three records)

ObjectiveAssessments define the subscore hierarchy — the landmarks on the map. The ClearPath Reading Benchmark has three subscores, each requiring one ObjectiveAssessment record. All three reference the same parent Assessment via assessmentReference. Each carries its own score range and performance level vocabulary.

CP-RD-PA — Phonological Awareness

[POST] /ed-fi/objectiveAssessments — CP-RD-PA

```
{

 "assessmentReference": {

  "assessmentIdentifier": "ClearPathBenchmark-Reading",
```

```
    "namespace": "uri://clearpath.example.com/assessment/clearpath-reading"
  },
  "identificationCode": "CP-RD-PA",
  "description": "Phonological Awareness subtest",
  "percentOfAssessment": 35,
  "scores": [
   {
    "assessmentReportingMethodDescriptor":
     "uri://clearpath.example.com/assessment/clearpath-reading/AssessmentReportingMethodDescriptor#Subtest
Score",
    "resultDatatypeTypeDescriptor": "uri://ed-fi.org/ResultDatatypeTypeDescriptor#Decimal",
    "minimumScore": "0.00",
    "maximumScore": "100.00"
   }
  ],
  "performanceLevels": [
   {
    "assessmentReportingMethodDescriptor":
     "uri://clearpath.example.com/assessment/clearpath-reading/AssessmentReportingMethodDescriptor#Subtest
Level",
    "performanceLevelDescriptor":
     "uri://clearpath.example.com/assessment/clearpath-reading/PerformanceLevelDescriptor#Lower than
Average",
    "resultDatatypeTypeDescriptor": "uri://ed-fi.org/ResultDatatypeTypeDescriptor#Level"
   },
   {
    "assessmentReportingMethodDescriptor":
     "uri://clearpath.example.com/assessment/clearpath-reading/AssessmentReportingMethodDescriptor#Subtest
Level",
    "performanceLevelDescriptor":
     "uri://clearpath.example.com/assessment/clearpath-reading/PerformanceLevelDescriptor#Average",
    "resultDatatypeTypeDescriptor": "uri://ed-fi.org/ResultDatatypeTypeDescriptor#Level"
   },
   {
```

```
    "assessmentReportingMethodDescriptor":
     "uri://clearpath.example.com/assessment/clearpath-reading/AssessmentReportingMethodDescriptor#Subtest Level",
    "performanceLevelDescriptor":
     "uri://clearpath.example.com/assessment/clearpath-reading/PerformanceLevelDescriptor#Higher than Average",
    "resultDatatypeTypeDescriptor": "uri://ed-fi.org/ResultDatatypeTypeDescriptor#Level"
   }
 ]
}
```

CP-RD-FL — Fluency

[POST] /ed-fi/objectiveAssessments — CP-RD-FL

```
{
 "assessmentReference": {
  "assessmentIdentifier": "ClearPathBenchmark-Reading",
  "namespace": "uri://clearpath.example.com/assessment/clearpath-reading"
 },
 "identificationCode": "CP-RD-FL",
 "description": "Fluency subtest",
 "percentOfAssessment": 35,
 "scores": [
  {
   "assessmentReportingMethodDescriptor":
    "uri://clearpath.example.com/assessment/clearpath-reading/AssessmentReportingMethodDescriptor#Subtest Score",
   "resultDatatypeTypeDescriptor": "uri://ed-fi.org/ResultDatatypeTypeDescriptor#Decimal",
   "minimumScore": "0.00",
   "maximumScore": "100.00"
  }
 ],
 "performanceLevels": [
  {
```

```
    "assessmentReportingMethodDescriptor":
     "uri://clearpath.example.com/assessment/clearpath-reading/AssessmentReportingMethodDescriptor#Subtest Level",
    "performanceLevelDescriptor":
     "uri://clearpath.example.com/assessment/clearpath-reading/PerformanceLevelDescriptor#Lower than Average",
    "resultDatatypeTypeDescriptor": "uri://ed-fi.org/ResultDatatypeTypeDescriptor#Level"
   },
   {
    "assessmentReportingMethodDescriptor":
     "uri://clearpath.example.com/assessment/clearpath-reading/AssessmentReportingMethodDescriptor#Subtest Level",
    "performanceLevelDescriptor":
     "uri://clearpath.example.com/assessment/clearpath-reading/PerformanceLevelDescriptor#Average",
    "resultDatatypeTypeDescriptor": "uri://ed-fi.org/ResultDatatypeTypeDescriptor#Level"
   },
   {
    "assessmentReportingMethodDescriptor":
     "uri://clearpath.example.com/assessment/clearpath-reading/AssessmentReportingMethodDescriptor#Subtest Level",
    "performanceLevelDescriptor":
     "uri://clearpath.example.com/assessment/clearpath-reading/PerformanceLevelDescriptor#Higher than Average",
    "resultDatatypeTypeDescriptor": "uri://ed-fi.org/ResultDatatypeTypeDescriptor#Level"
   }
  ]
}
```


CP-RD-VC — Vocabulary

[POST] /ed-fi/objectiveAssessments — CP-RD-VC


```
{
  "assessmentReference": {
   "assessmentIdentifier": "ClearPathBenchmark-Reading",
   "namespace": "uri://clearpath.example.com/assessment/clearpath-reading"
```

```
  },
  "identificationCode": "CP-RD-VC",
  "description": "Vocabulary subtest",
  "percentOfAssessment": 30,
  "scores": [
   {
    "assessmentReportingMethodDescriptor":
     "uri://clearpath.example.com/assessment/clearpath-reading/AssessmentReportingMethodDescriptor#Subtest
Score",
    "resultDatatypeTypeDescriptor": "uri://ed-fi.org/ResultDatatypeTypeDescriptor#Decimal",
    "minimumScore": "0.00",
    "maximumScore": "100.00"
   }
  ],
  "performanceLevels": [
   {
    "assessmentReportingMethodDescriptor":
     "uri://clearpath.example.com/assessment/clearpath-reading/AssessmentReportingMethodDescriptor#Subtest
Level",
    "performanceLevelDescriptor":
     "uri://clearpath.example.com/assessment/clearpath-reading/PerformanceLevelDescriptor#Lower than
Average",
    "resultDatatypeTypeDescriptor": "uri://ed-fi.org/ResultDatatypeTypeDescriptor#Level"
   },
   {
    "assessmentReportingMethodDescriptor":
     "uri://clearpath.example.com/assessment/clearpath-reading/AssessmentReportingMethodDescriptor#Subtest
Level",
    "performanceLevelDescriptor":
     "uri://clearpath.example.com/assessment/clearpath-reading/PerformanceLevelDescriptor#Average",
    "resultDatatypeTypeDescriptor": "uri://ed-fi.org/ResultDatatypeTypeDescriptor#Level"
   },
   {
    "assessmentReportingMethodDescriptor":
```

```
    "uri://clearpath.example.com/assessment/clearpath-reading/AssessmentReportingMethodDescriptor#Subtest Level",
    "performanceLevelDescriptor":
    "uri://clearpath.example.com/assessment/clearpath-reading/PerformanceLevelDescriptor#Higher than Average",
    "resultDatatypeTypeDescriptor": "uri://ed-fi.org/ResultDatatypeTypeDescriptor#Level"
  }
 ]
}
```

GOVERNANCE — SECTION 5: HIERARCHY MUST MIRROR THE SCORE REPORT

The ObjectiveAssessment hierarchy must exactly mirror the structure of ClearPath's score report. Three subscores reported by the vendor means exactly three ObjectiveAssessment records — not fewer (which would collapse the hierarchy), not more (which would invent structure that does not exist on the score report). The identificationCode value is the stable key for each subtest and must not change between school years.

Step 3 — POST /ed-fi/studentAssessments

The StudentAssessment records the event — one record per student per administration. This example shows fake_student_1, a Kindergartener, taking the Reading Benchmark in the fall window on January 1, 2024. All event context fields are required: administrationDate, schoolYearTypeReference, whenAssessedGradeLevelDescriptor, and assessmentPeriodDescriptor. Together they form the event identity anchor for longitudinal tracking.

[POST] /ed-fi/studentAssessments — fake_student_1, Reading, Fall 2024

```
{
 "studentAssessmentIdentifier": "fake_student_1-ClearPathBenchmark-Reading-2024-Fall",
 "assessmentReference": {
  "assessmentIdentifier": "ClearPathBenchmark-Reading",
  "namespace": "uri://clearpath.example.com/assessment/clearpath-reading"
 },
 "studentReference": {
  "studentUniqueId": "1000"
 },
 "schoolYearTypeReference": {
```

```
    "schoolYear": 2024
  },
  "administrationDate": "2024-01-01",
  "administrationEndDate": "2024-01-01",
  "whenAssessedGradeLevelDescriptor": "uri://ed-fi.org/GradeLevelDescriptor#Kindergarten",
  "assessmentPeriodDescriptor":
    "uri://clearpath.example.com/assessment/clearpath-reading/AssessmentPeriodDescriptor#Fall",
  "scoreResults": [
    {
      "assessmentReportingMethodDescriptor":
        "uri://clearpath.example.com/assessment/clearpath-
reading/AssessmentReportingMethodDescriptor#Composite Score",
      "resultDatatypeTypeDescriptor": "uri://ed-fi.org/ResultDatatypeTypeDescriptor#Integer",
      "result": "100"
    },
    {
      "assessmentReportingMethodDescriptor":
        "uri://clearpath.example.com/assessment/clearpath-reading/AssessmentReportingMethodDescriptor#SEM",
      "resultDatatypeTypeDescriptor": "uri://ed-fi.org/ResultDatatypeTypeDescriptor#Integer",
      "result": "10"
    }
  ],
  "performanceLevels": [
    {
      "assessmentReportingMethodDescriptor":
        "uri://clearpath.example.com/assessment/clearpath-
reading/AssessmentReportingMethodDescriptor#Benchmark Level",
      "performanceLevelDescriptor":
        "uri://clearpath.example.com/assessment/clearpath-reading/PerformanceLevelDescriptor#Did not yet meet
expectations",
      "performanceLevelMet": true
    }
  ],
```

```
  "studentObjectiveAssessments": [
    // see Step 4 for full collection
  ]
}
```

GOVERNANCE — SECTION 6: EVENT IDENTITY AND LONGITUDINAL INTEGRITY

The natural key for this record is the combination of studentUniqueId + assessmentIdentifier + namespace + administrationDate + schoolYear. Every field must be deterministic and stable across all ingestion runs. If administrationDate changes between runs for the same test event — for example, a processing timestamp is substituted — the system will create a duplicate record instead of updating the original, resulting in doubled student data and broken longitudinal analysis.

STUDENT IDENTITY — SECTION 8

The studentUniqueId value 1000 is the Ed-Fi state identifier for this student, resolved from ClearPath's internal vendor identifier fake_student_1 via the identity crosswalk at integration time. The vendor's internal identifier must never be placed in studentUniqueId. Correct identity resolution at this step determines whether every downstream assessment record links accurately to the right student enrollment, demographic, and program data.

COMMON ERROR — SECTION 5.4: SCORES VS. PERFORMANCE LEVELS

Scores and performance levels are separate arrays serving distinct analytical purposes. scoreResults carries numeric values (Composite Score = 100, SEM = 10). performanceLevels carries categorical classification (Did not yet meet expectations). Never place a performance level label inside scoreResults, and never place a numeric score value inside performanceLevels. Both errors produce data that is analytically unusable and will fail validation.

Step 4 — studentObjectiveAssessments collection (nested in StudentAssessment)

The studentObjectiveAssessments array is nested inside the StudentAssessment body — it is not a separate API call. Each element references one ObjectiveAssessment by its identificationCode and carries that subtest's score result and performance level. All three subtest results for fake_student_1 are shown below with values drawn from the sample source data.

Nested array — studentObjectiveAssessments inside StudentAssessment body (not a separate POST)

```
// studentObjectiveAssessments array -- nested inside StudentAssessment body
```

```
[
 {
  "objectiveAssessmentReference": {
   "assessmentIdentifier": "ClearPathBenchmark-Reading",
   "identificationCode": "CP-RD-PA",
   "namespace": "uri://clearpath.example.com/assessment/clearpath-reading"
  },
  "scoreResults": [
   {
    "assessmentReportingMethodDescriptor":
     "uri://clearpath.example.com/assessment/clearpath-reading/AssessmentReportingMethodDescriptor#Subtest
Score",
    "resultDatatypeTypeDescriptor": "uri://ed-fi.org/ResultDatatypeTypeDescriptor#Decimal",
    "result": "20.10"
   }
  ],
  "performanceLevels": [
   {
    "assessmentReportingMethodDescriptor":
     "uri://clearpath.example.com/assessment/clearpath-reading/AssessmentReportingMethodDescriptor#Subtest
Level",
    "performanceLevelDescriptor":
     "uri://clearpath.example.com/assessment/clearpath-reading/PerformanceLevelDescriptor#Lower than
Average",
    "performanceLevelMet": true
   }
  ]
 },
 {
  "objectiveAssessmentReference": {
   "assessmentIdentifier": "ClearPathBenchmark-Reading",
   "identificationCode": "CP-RD-FL",
   "namespace": "uri://clearpath.example.com/assessment/clearpath-reading"
```

```
  },
  "scoreResults": [
   {
    "assessmentReportingMethodDescriptor":
     "uri://clearpath.example.com/assessment/clearpath-reading/AssessmentReportingMethodDescriptor#Subtest
Score",
    "resultDatatypeTypeDescriptor": "uri://ed-fi.org/ResultDatatypeTypeDescriptor#Decimal",
    "result": "20.20"
   }
  ],
  "performanceLevels": [
   {
    "assessmentReportingMethodDescriptor":
     "uri://clearpath.example.com/assessment/clearpath-reading/AssessmentReportingMethodDescriptor#Subtest
Level",
    "performanceLevelDescriptor":
     "uri://clearpath.example.com/assessment/clearpath-reading/PerformanceLevelDescriptor#Lower than
Average",
    "performanceLevelMet": true
   }
  ]
 },
 {
  "objectiveAssessmentReference": {
   "assessmentIdentifier": "ClearPathBenchmark-Reading",
   "identificationCode": "CP-RD-VC",
   "namespace": "uri://clearpath.example.com/assessment/clearpath-reading"
  },
  "scoreResults": [
   {
    "assessmentReportingMethodDescriptor":
     "uri://clearpath.example.com/assessment/clearpath-reading/AssessmentReportingMethodDescriptor#Subtest
Score",
    "resultDatatypeTypeDescriptor": "uri://ed-fi.org/ResultDatatypeTypeDescriptor#Decimal",
```

```
      "result": "20.20"
    }
  ],
  "performanceLevels": [
    {
      "assessmentReportingMethodDescriptor":
       "uri://clearpath.example.com/assessment/clearpath-reading/AssessmentReportingMethodDescriptor#Subtest
Level",
      "performanceLevelDescriptor":
       "uri://clearpath.example.com/assessment/clearpath-reading/PerformanceLevelDescriptor#Lower than
Average",
      "performanceLevelMet": true
    }
  ]
 }
]
```


GOVERNANCE — SECTION 5.2: CORRECT GRAIN ENFORCEMENT

Overall composite results belong on StudentAssessment.scoreResults and StudentAssessment.performanceLevels. Subtest results belong in the studentObjectiveAssessments collection, each nested under its corresponding objectiveAssessmentReference. Placing a composite score inside a studentObjectiveAssessment element — or placing a subtest score directly on StudentAssessment — violates grain enforcement. Both errors produce analytically unusable data and will fail certification validation.

API SEQUENCING — SECTION 11

Resources must be POSTed in dependency order: Assessment first, then ObjectiveAssessments (which reference the Assessment), then StudentAssessments (which reference both). Posting a StudentAssessment before its parent Assessment exists will fail with a referential integrity error. This dependency order must be deterministic and repeatable across every ingestion run.

Descriptor reference — ClearPath Reading Benchmark

Every descriptor used in this example is listed below with its namespace, type, and ownership classification. Vendor-owned descriptors use the clearpath.example.com namespace prefix. Ed-Fi

shared descriptors use the ed-fi.org namespace. This table should be included in vendor documentation and supplied as part of the integration evidence artifact package.

Descriptor value	Type	Owner
AssessmentReportingMethodDescriptor#Composite Score	AssessmentReportingMethod	**Vendor**
AssessmentReportingMethodDescriptor#SEM	AssessmentReportingMethod	**Vendor**
AssessmentReportingMethodDescriptor#Benchmark Level	AssessmentReportingMethod	**Vendor**
AssessmentReportingMethodDescriptor#Subtest Score	AssessmentReportingMethod	**Vendor**
AssessmentReportingMethodDescriptor#Subtest Level	AssessmentReportingMethod	**Vendor**
AssessmentPeriodDescriptor#Fall	AssessmentPeriod	**Vendor**
PerformanceLevelDescriptor#Did not yet meet expectations	PerformanceLevel (composite)	**Vendor**
PerformanceLevelDescriptor#Approaching expectations	PerformanceLevel (composite)	**Vendor**
PerformanceLevelDescriptor#Met expectations	PerformanceLevel (composite)	**Vendor**
PerformanceLevelDescriptor#Exceeded expectations	PerformanceLevel (composite)	**Vendor**
PerformanceLevelDescriptor#Lower than Average	PerformanceLevel (subtest)	**Vendor**
PerformanceLevelDescriptor#Average	PerformanceLevel (subtest)	**Vendor**
PerformanceLevelDescriptor#Higher than Average	PerformanceLevel (subtest)	**Vendor**
GradeLevelDescriptor#Kindergarten (and Grade 1–3)	GradeLevel	**Ed-Fi shared**
AcademicSubjectDescriptor#English Language Arts	AcademicSubject	**Ed-Fi shared**
ResultDatatypeTypeDescriptor#Integer	ResultDatatypeType	**Ed-Fi shared**
ResultDatatypeTypeDescriptor#Decimal	ResultDatatypeType	**Ed-Fi shared**

ResultDatatypeTypeDescriptor#Level	ResultDatatypeType	**Ed-Fi shared**

GOVERNANCE — SECTION 7: DESCRIPTOR AND NAMESPACE GOVERNANCE

Vendor-owned descriptors must use the vendor's registered namespace prefix. They must not be normalized, translated, or replaced with Ed-Fi default namespace values at ingestion. A state or district implementer is not permitted to change PerformanceLevelDescriptor#Did not yet meet expectations to a locally preferred label during load. Doing so destroys the semantic integrity that enables cross-vendor and cross-state comparability. Shared cross-domain descriptors (GradeLevel, AcademicSubject, ResultDatatypeType) use the Ed-Fi default namespace and may be configured through governed mapping layers.

API call sequence summary

All resources must be POSTed in the dependency order shown below. Steps 1 and 2 are one-time setup for the assessment definition and are re-posted only when the assessment version changes. Step 3 is executed once per student per administration window.

#	Resource	Records	Dependency
1	POST /ed-fi/assessments	1	None
2	POST /ed-fi/objectiveAssessments	3 (one per subscore)	Assessment (Step 1) must exist
3	POST /ed-fi/studentAssessments	1 per student per window	Assessment + ObjectiveAssessments (Steps 1 and 2) must exist

REPROCESSING — SECTION 11.3

All three resource types are safe to reprocess. A repeated POST with the same natural key will upsert rather than duplicate, provided all identity fields remain identical across runs. Never substitute a processing timestamp for administrationDate — doing so breaks idempotent reprocessing and will generate duplicate student records that cannot be deduplicated without manual intervention.

Part III – Data Modeling Requirements

3. Assessment Identity

Assessment identity must be established and consistently applied across implementations. Assessment identity is a governance decision, not an implementation detail.

3.1 What Makes an Assessment Unique?

An Assessment is uniquely defined by the combination of Namespace and AssessmentIdentifier. Together, these form the identity contract that determines analytic grain, ownership, and comparability across implementations and over time. Namespace establishes ownership and prevents collisions, while AssessmentIdentifier defines meaning and analytic grain.

When identity is inconsistently defined:

- The same assessment may appear at different grains
- Subject meaning may be ambiguous
- Longitudinal results may fragment
- Downstream systems must rely on vendor-specific logic to interpret the data

This breaks interoperability and undermines one of Ed-Fi's core goals: eliminating custom transformation logic.

A consistent identity strategy ensures that:

- A math benchmark and a reading benchmark are represented as distinct assessments
- The same assessment across years retains a stable identifier when the structure has not changed
- Form or administration differences are handled outside of identity unless they change the meaning of the score

Assessment Provider Responsibility

The assessment provider is responsible for ensuring that assessment identity is consistently defined, applied, and governed across implementations:

- Defining the identifier convention
- Ensuring consistency across implementations
- Aligning identity decisions with governance expectations

Prohibited Patterns

The following patterns are not allowed because they introduce ambiguity in identity and require downstream systems to reconstruct meaning:

- Encoding subject inconsistently (sometimes in the identifier, sometimes not)
- Collapsing multiple subjects into a single top-level assessment
- Treating forms or versions inconsistently across implementations
- Encoding time elements (BOY, MOY, EOY, school year) into the identifier when the structure is unchanged

Establishing a consistent identity strategy ensures that assessment results remain interpretable, comparable across implementations, and usable for longitudinal analysis without requiring downstream systems to infer or reconstruct meaning through custom logic.

3.2 The Highest-Grain Rule

The AssessmentIdentifier must represent the highest grain at which overall results are defined. This means the identifier defines the level at which a student receives a complete, top-level outcome for the assessment. Assessment identity must align with how results are actually reported and interpreted. The highest grain is the level at which overall outcomes, such as composite scores, overall performance levels, or total scores, are defined.

If identity is defined below this level:

- Overall results may be fragmented across multiple assessments
- Relationships between subscores and overall outcomes may be lost
- Downstream systems must reconstruct hierarchy to interpret results

If identity is defined above this level:

- Multiple distinct results may be incorrectly combined
- Subject meaning may become ambiguous
- Analytic comparisons become unreliable

Defining identity at the correct grain ensures that the structure of results aligns with how they are consumed in reporting and analysis.

In practice, the highest grain should align with the assessment title and subject, because this is the level at which overall results are defined and interpreted.

For example:

- A math benchmark and a reading benchmark should be represented as separate assessments
- A multi-subject instrument should not collapse multiple subjects into a single top-level assessment unless the vendor explicitly reports a composite at that level

Subscores, strands, domains, and other breakdowns must not define separate assessments. These belong within the assessment structure and are represented through ObjectiveAssessment and the StudentObjectiveAssessment collection within the StudentAssessment record.

Assessment Provider Responsibility

The assessment provider is responsible for ensuring that assessment identity is defined at the correct grain and aligned to how results are reported and interpreted:

- Aligning identifier design to the vendor's score report structure
- Ensuring consistency across implementations
- Avoiding adjustments to grain based on partner-specific requirements

The chosen grain must remain stable and must not vary across deployments.

Prohibited Patterns

The following patterns are not allowed because they break alignment between identity and results by distorting the relationship between structure and outcomes and requiring downstream systems to reconstruct meaning:

- Defining separate assessments for subscores, strands, or domains
- Collapsing multiple subjects into a single assessment when results are reported separately
- Defining identity at a level that does not include overall results

Defining identity at the correct grain ensures that downstream systems can support growth analysis, subgroup reporting, and cross-assessment comparisons without reconstructing meaning through custom logic.

3.3 Identifier Stability

AssessmentIdentifier values must remain stable over time for the same assessment when the structure and meaning of results have not changed. This stability is foundational to longitudinal analysis, ensuring that results can be compared across administrations, school years, and implementations.

When identifiers change unnecessarily:

- Longitudinal trends are broken
- Historical results cannot be reliably compared
- Duplicate assessments appear in downstream systems
- Analytics require custom logic to reconcile identity over time

Identifier stability means that:

- The same assessment administered across multiple years retains the same AssessmentIdentifier when the structure and meaning of results remain consistent
- Routine operational differences, such as administration window (BOY, MOY, EOY), school year, or minor form variations, do not require a new identifier

A new AssessmentIdentifier is required when:

- The assessment structure changes (e.g., new hierarchy, different scoring model)
- The meaning of results changes (e.g., scale reset, redefinition of performance levels)
- The assessment is redesigned in a way that affects interpretation

AssessmentIdentifier must not include:

- Vendor name (Namespace already communicates ownership)
- Administration window (BOY, MOY, EOY)
- Random or unstable numeric codes that are not interpretable
- State labels unless the assessment is structurally different in that state in a way that affects scoring, hierarchy, or interpretation

Version indicators should only be included when the assessment has materially changed in a way that affects comparability or interpretation.

Assessment Provider Responsibility

The assessment provider is responsible for ensuring that assessment identifiers remain stable over time and change only when the structure or meaning of results materially changes.

- Maintaining identifier stability
- Defining clear versioning rules
- Establishing criteria for when a new identifier is required
- Ensuring that identifier decisions are applied consistently across implementations
- Aligning identifier changes with governance expectations and documentation

Prohibited Patterns

The following patterns are not allowed because they break longitudinal integrity by fragmenting assessment identity and requiring downstream systems to reconstruct meaning:

- Changing AssessmentIdentifier values for routine administration differences
- Encoding time-based elements or versioning information directly into the identifier when the assessment meaning has not changed
- Reusing an AssessmentIdentifier when the assessment structure or meaning has materially changed
- Creating new identifiers for the same assessment across different implementations or partners

Maintaining identifier stability ensures that longitudinal analysis, growth measurement, and cross-year comparisons remain accurate and do not require downstream systems to reconcile fragmented assessment identities through custom logic.

3.4 Assessment Family

AssessmentFamily is used to group related assessments that are part of the same product line or progression but remain distinct at the assessment identity level.

Assessments within a family may share common characteristics, such as subject area, vendor origin, or intended use, but each assessment must maintain its own Namespace and AssessmentIdentifier based on its structure and results. AssessmentFamily does not replace or override assessment identity. It provides an organizational relationship across assessments that are intentionally modeled as separate.

Assessment families are especially useful for representing related instruments within a vendor product suite.

However, these relationships must not be used to collapse or blur distinctions between assessments that produce different results or have different structures.

A correct use of AssessmentFamily ensures that:

- Each assessment retains a clear and consistent identity aligned to its results
- Relationships between assessments are explicitly defined without altering their meaning
- Grouping supports organization and navigation without impacting analytic interpretation

Assessment Provider Responsibility

The assessment provider is responsible for ensuring that assessment family relationships are clearly defined, consistently applied, and do not override or conflict with assessment identity:

- Establishing clear criteria for which assessments belong to the same family
- Ensuring consistent grouping across implementations
- Aligning family definitions with governance expectations and documentation

AssessmentFamily must not be used as a substitute for proper identity design or to compensate for inconsistent identifier strategies.

Prohibited Patterns

The following patterns are not allowed because they obscure assessment meaning and create ambiguity in how results should be interpreted:

- Using AssessmentFamily to group assessments that should be represented as a single assessment
- Using AssessmentFamily to compensate for inconsistent or incorrect AssessmentIdentifier definitions
- Grouping assessments with materially different structures or scoring models without a clear distinction
- Relying on AssessmentFamily to drive analytic logic instead of using a properly defined assessment identity

Proper use of AssessmentFamily ensures that related assessments can be organized and understood together while preserving a clear, consistent identity. This allows downstream systems to group, filter, and analyze related assessments without compromising interpretability or requiring custom logic to reconstruct relationships.

4. Subject Strategy

Subject strategy exists to enforce the foundational principle of protecting subject integrity. Subject ambiguity is one of the highest-cost failure patterns because it forces downstream systems to infer meaning using vendor-specific logic, which is not stable or scalable.

4.1 The Single Subject Rule

Each Assessment must resolve to exactly one AcademicSubject.

This is the standard that enables district analytics, state reporting alignment, course-linked reporting, CCR and CCMR pipelines, and cross-vendor comparability. When subject is ambiguous at the top level, downstream systems must infer meaning from score names or structure, which introduces inconsistency and breaks comparability.

If an assessment cannot resolve to a single subject, it must use a subject of *Composite*, with subject-specific results represented through ObjectiveAssessments.

Assessment Provider Responsibility

The assessment provider is responsible for ensuring that each assessment resolves to a single, explicitly defined subject that aligns with how results are reported and interpreted:

- Defining a single AcademicSubject for each assessment
- Ensuring subject assignment is consistent across implementations
- Avoiding reliance on downstream systems to infer subject meaning
- Aligning subject decisions with governance expectations

Prohibited Patterns

The following patterns are not allowed because they introduce subject ambiguity and require downstream systems to reconstruct meaning:

- Assigning multiple subjects to a single top-level Assessment
- Requiring downstream systems to infer subject from score names or objective structure
- Inconsistently assigning subject across implementations for the same assessment

Establishing a single, explicit subject ensures that assessment results remain interpretable, comparable across implementations, and usable for subject-based analysis without requiring downstream systems to infer meaning through custom logic.

4.2 Subject Integrity Enforcement

Subject integrity must be established at ingestion and must not depend on downstream interpretation. If subject meaning is not explicit in the model, the data may load successfully but will not support reliable analytics.

Subject integrity ensures that:

- Results can be filtered and compared consistently
- Cross-vendor analysis remains valid
- Subject-based reporting does not require custom logic
- Composite and subject-specific results remain analytically distinct

Composite Handling

Cross-subject instruments are the primary case where subject integrity must be enforced through structure.

For any assessment that spans multiple subjects:

- The top-level Assessment subject must be Composite
- Subject-specific results must be represented through ObjectiveAssessment

This approach ensures that:

- Composite results are not misattributed to a single subject
- Subject-level results remain available for analysis
- Subject-based queries behave predictably

Subject meaning must be clear at every level of the model. A subject-specific query should not unintentionally include composite results due to ambiguous modeling.

Assessment Provider Responsibility

The assessment provider is responsible for ensuring that subject meaning is explicit, stable, and consistently applied across all aspects of the assessment model.

- Assign Composite as the top-level subject for cross-subject assessments
- Ensuring subject meaning is explicitly defined at the assessment level
- Preventing ambiguity between composite and subject-specific results
- Maintaining consistency across implementations and over time
- Aligning subject modeling decisions with governance expectations

Prohibited Patterns

The following patterns are not allowed because they break subject integrity and introduce ambiguity in interpretation:

- Multi-subject top-level assessments without Composite designation
- Any design that requires subject inference from score names or structure
- Inconsistent subject modeling across implementations of the same assessment

Enforcing subject integrity at ingestion ensures that assessment data remains reliable, comparable, and analytically usable across systems, preventing ambiguity and eliminating the need for downstream systems to infer or reconstruct subject meaning.

5. Hierarchy and Results Placement

Hierarchy and results placement define how assessment structure (metadata) and student outcomes (results) are represented in the Ed-Fi model.

This section operationalizes the core entities defined in Section 2.2:

- Assessment and ObjectiveAssessment define the structure
- StudentAssessment and StudentObjectiveAssessment deliver results within that structure

These two layers must remain strictly aligned. The structure declared by the assessment must match the placement of student results.

This is the primary enforcement point for a native Ed-Fi assessment integration.

Correct implementation preserves the relationship between structure and results exactly as defined in the vendor score report. This allows results to be interpreted directly, without vendor-specific logic, reconstruction, or score-name parsing.

Incorrect implementation breaks that relationship. Data may load into Ed-Fi, but:

- Hierarchy must be inferred rather than represented
- Subject and grain become ambiguous
- Downstream systems must reconstruct the structure
- Cross-vendor comparability is lost

This section defines the rules that eliminate these failure modes by ensuring that:

- The full score report is represented
- Hierarchy is modeled explicitly and completely
- Results are placed at the correct grain
- Structure and results remain aligned across all implementations

These rules are not optional. They are required for an integration to be considered complete, interoperable, and analytically usable.

5.1 Modeling the Hierarchy

The assessment hierarchy must faithfully and completely represent the structure defined in the vendor score report.

This is where the conceptual model from Section 2.2 becomes operational:

- *Assessment* defines the top-level instrument at the highest grain where overall results exist
- *ObjectiveAssessment* defines the hierarchical components of the assessment
- *StudentAssessment* records a student's attempt and holds the overall results
- *StudentObjectiveAssessment* (a collection within *StudentAssessment*) records results aligned to ObjectiveAssessment

Assessment metadata must declare the structure first. Student results must then align to that structure exactly. A native integration is not simply sending scores. It is sending a structured representation of the assessment and results that conform to that structure.

If the vendor score report includes subscores, domains, strands, subtests, measures, skills, or reporting categories, those structures must be represented using ObjectiveAssessment. The corresponding results must be delivered through StudentObjectiveAssessment. If the score report contains hierarchy, the model must reflect it exactly.

When hierarchy is not faithfully modeled:

- Structural meaning is lost
- Subscore relationships become ambiguous
- Results must be interpreted through score names or vendor-specific conventions
- Downstream systems must reconstruct the structure
- Cross-vendor comparison becomes unreliable

Assessment Provider Responsibility

The assessment provider is responsible for ensuring that the assessment hierarchy is faithfully defined, consistently applied, and aligned to the vendor score report across implementations:

- Define Assessment at the correct top-level grain
- Model ObjectiveAssessment structures wherever hierarchy exists
- Apply recursive hierarchy when multiple levels are present
- Preserve all parent-child relationships across levels
- Ensure StudentObjectiveAssessment results align with ObjectiveAssessment definitions
- Maintain consistent hierarchy across implementations

Prohibited Patterns

The following patterns are not allowed because they break the relationship between assessment structure and results and require downstream systems to reconstruct hierarchy or infer meaning:

- Omitting ObjectiveAssessment when hierarchy exists
- Collapsing multi-level hierarchy into a single level
- Flattening structure into score names instead of modeling it
- Defining ObjectiveAssessment structures that do not match the score report
- Defining structure without delivering corresponding results
- Treating StudentObjectiveAssessment as a standalone entity or payload
- Modeling the same assessment with different hierarchies across implementations

Modeling the true hierarchy ensures that structure and results remain aligned and eliminates the need for downstream reconstruction.

5.2 Full Result Delivery

A native integration must deliver the complete vendor score report, not a subset of results.

All results present in the vendor score report must be represented and placed at the correct level unless prohibited by policy or regulation. Partial representation is not a complete or native integration. This is the primary enforcement point for correct modeling.

Core Principle

Every reported value must be delivered:

- If a value appears on the vendor score report, it must appear in the integration
- If a structure is defined (via ObjectiveAssessment), corresponding results must be present
- If results exist, they must be delivered at the level where they are reported

A native integration is a faithful transmission of the full score report, not an interpretation or reduction of it.

What Must Be Included

The integration must include all available result types, including:

- Scale scores
- Performance levels
- Percentiles and rankings
- Growth Measures
- Subscores at all reported levels
- Subtest, domain, strand, or skill-level results
- And additional reported metrics or indicators

If the vendor reports it, the integration must deliver it.

Assessment Provider Responsibility

The assessment provider is responsible for ensuring that the full score report is delivered consistently and completely across implementations:

- Deliver all reported results, not a subset
- Ensure every defined structure has corresponding results
- Preserve all reported values and levels of detail
- Maintain consistency across administrations and integrations

Prohibited Patterns

The following patterns are not allowed because they result in incomplete or distorted representations of the score report:

- Omitting subscores or lower-level results
- Sending only overall scores when additional detail exists
- Delivering different levels of detail across implementations
- Defining structure without delivering corresponding results
- Filtering results based on perceived importance or use

- Delivering derived values instead of vendor-reported values

5.3 Recursive Objective Structures

ObjectiveAssessment must be used recursively when the score report includes multiple levels of hierarchy.

Many assessments include nested structures such as domains, strands, subdomains, or reporting categories. These must be represented using explicit parent-child ObjectiveAssessment relationships.

All results across these levels must be captured within the StudentObjectiveAssessment collection.

The rule is simple: *If the score report has hierarchy, the model must reflect it.*

When recursive structure is not modeled:

- Multi-level relationships are lost
- Subscores lose context
- Results cannot be interpreted consistently
- Detailed analysis becomes unreliable

Assessment Provider Responsibility

The assessment provider is responsible for ensuring that recursive objective structures are accurately defined, fully represented, and consistently applied across implementations:

- Identify all levels of hierarchy in the score report
- Define ObjectiveAssessment relationships to match that hierarchy
- Model all levels consistently
- Align results to the correct level
- Ensure all results are delivered within StudentObjectiveAssessment

Prohibited Patterns

The following patterns are not allowed because they collapse or distort hierarchical structure and require downstream systems to reconstruct relationships or infer meaning:

- Modeling multi-level assessments as flat structures
- Skipping levels present in the score report
- Representing hierarchy through naming instead of structure
- Creating structure without corresponding results
- Inconsistent hierarchy across implementations

Recursive modeling ensures that complex assessments retain their full meaning without downstream reconstruction.

5.4 Score vs. Performance Level Separation

Scores and performance levels are distinct and must be modeled separately.

- *Scores* are quantitative (scale scores, raw scores, percentiles, growth metrics)
- *Performance levels* are categorical (proficiency levels, risk bands, achievement categories)

They are not interchangeable and are not inherently one-to-one.

When these concepts are conflated:

- Numeric values are misinterpreted as categories
- Performance levels are compared incorrectly across assessments
- Analytical meaning is lost

Assessment Provider Responsibility

The assessment provider is responsible for ensuring that scores and performance levels are clearly separated, correctly represented, and consistently applied across implementations:

- Model quantitative results as scores
- Represent performance levels using descriptor-based fields
- Preserve vendor-defined performance level semantics
- Avoid implicit mappings between scores and levels
- Use descriptor categories consistently

Prohibited Patterns

The following patterns are not allowed because they conflate distinct concepts and require downstream systems to reinterpret or infer analytical meaning:

- Storing performance levels as numeric scores
- Storing scores as categorical descriptors
- Treating scores and performance levels as interchangeable
- Reusing descriptor categories across contexts
- Embedding performance level meaning in score values

Separating scores from performance levels preserves analytical clarity and prevents downstream misinterpretation.

5.5 Indicators vs. Scores

Not all assessment data represents performance. Indicators must not be modeled as scores.

Indicators include:

- Participation status
- Accommodations
- Testing conditions
- Risk or context flags

These provide context, not performance outcomes.

Indicators must be represented using other appropriate fields or constructs within the student assessment entity and must not be included in score results.

When indicators are stored as scores:

- Performance data is contaminated
- Non-performance values are misinterpreted
- Analytics produce incorrect conclusions

Ed-Fi Data Standard 6.0 introduces structured support for indicators not otherwise captured in a specific property, reinforcing this separation.

Assessment Provider Responsibility

The assessment provider is responsible for ensuring that indicators are correctly represented, separated from performance data, and consistently applied across implementations:

- Identify non-performance attributes
- Map indicators to appropriate constructs
- Ensure indicators are not included in score results
- Preserve the distinction between performance and context
- Align indicator modeling with governance expectations

Prohibited Patterns

The following patterns are not allowed because they misrepresent contextual data as performance and require downstream systems to reinterpret or infer meaning:

- Storing indicators as score results
- Using score fields for participation or status
- Mixing indicators with performance data
- Encoding indicator meaning through score naming

Separating indicators from scores ensures that performance data remains accurate and analytically valid.

6. Event Identity and Longitudinal Integrity

Event identity defines how a student's assessment attempt is anchored in time and context. Without a complete and consistent event definition, assessment results cannot be reliably interpreted, aligned to enrollment, or used for longitudinal analysis.

A StudentAssessment is not just a container for results. It represents a specific assessment event for a student. That event must include the contextual fields required to distinguish when the assessment occurred, under what conditions, and how it relates to other attempts over time.

This section enforces the foundational principle of including full event context. Without it, results may load successfully, but cannot support growth analysis, cohort tracking, enrollment alignment, or reporting pipelines such as CCR and CCMR.

6.1 Required Event Context Fields

A StudentAssessment must include the event context necessary to uniquely identify and interpret a student's assessment attempt.

These fields anchor the assessment in time and context and must be populated when available, even if not required by a state reporting specification.

Required fields that serve a distinct purpose:

- *SchoolYear* ensures that results are grouped into the correct academic year

- *AdministrationDate* distinguishes individual attempts and enables alignment to enrollment and instruction timelines
- *AssessmentPeriod* provides a consistent interpretation of when the assessment occurred within the instructional cycle
- *WhenAssessedGradeLevel* anchors the result to the student's grade at the time of assessment
- *RetestIndicator* distinguishes first attempts from subsequent attempts when multiple administrations occur

Without these fields, assessment results lose temporal meaning and cannot be reliably interpreted across systems.

Assessment Provider Responsibility

The assessment provider is responsible for ensuring that event context is fully defined, accurately represented, and consistently applied across implementations:

- Populate all required event context fields when available
- Ensure event context reflects the actual timing and conditions of the assessment
- Align event context values to the vendor's source data and score report
- Maintain consistent use of event context fields across implementations
- Ensure multiple attempts are distinguishable through event context rather than identity

Prohibited Patterns

The following patterns are not allowed because they remove temporal context and require downstream systems to reconstruct or infer event identity:

- Omitting SchoolYear when it is available
- Omitting AdministrationDate when it is available
- Omitting AssessmentPeriod when the vendor defines one
- Omitting WhenAssessedGradeLevel when available
- Failing to distinguish multiple attempts when retesting occurs

Populating a complete event context ensures that assessment results remain interpretable, align with enrollment and reporting periods, and support longitudinal analysis without requiring downstream systems to infer timing or reconstruct attempt history.

6.2 Why Missing Event Context Breaks Everything

Event context is not optional metadata. It is the foundation for interpreting results over time. When event context is missing or incomplete:

- Attempts cannot be reliably grouped into the correct academic year
- Multiple attempts cannot be distinguished or sequenced correctly
- Enrollment alignment becomes unreliable or impossible
- Growth analysis becomes inconsistent across systems
- Cohort and program analysis cannot be trusted

The impact is immediate and systemic:

- Without *SchoolYear*, off-cycle or summer assessments may be grouped into the wrong reporting window

- Without *AdministrationDate*, retakes cannot be ordered, and attempt history becomes unreliable
- Without *AssessmentPeriod* and grade context, systems must invent heuristics for growth interpretation, leading to inconsistent results
- Without a clear attempt context, longitudinal grouping becomes fragmented

The result is not just incomplete data. It is ambiguous data that cannot be consistently interpreted across systems, partners, or time.

Assessment Provider Responsibility

The assessment provider is responsible for ensuring that event identity is complete and sufficient to support longitudinal analysis and consistent interpretation across implementations:

- Provide complete event context for every StudentAssessment record
- Ensure that event context enables reliable grouping, sequencing, and comparison of results
- Align event context to enrollment and reporting timelines
- Maintain consistency in how event context is defined and populated across implementations

Prohibited Patterns

The following patterns are not allowed because they break longitudinal integrity and require downstream systems to reconstruct or infer event meaning:

- Delivering results without sufficient context to distinguish attempts
- Relying on downstream systems to infer academic year or reporting period
- Using inconsistent or ambiguous definitions of assessment period
- Providing incomplete event context that prevents reliable grouping or sequencing of results

Ensuring complete event identity preserves longitudinal integrity, enabling accurate growth analysis, cohort tracking, and reporting without requiring downstream systems to reconstruct or infer temporal context.

6.3 Event Identity Modeling Rules

Event identity must be represented through explicit context fields, not encoded into identifiers or implied through naming conventions.

Identifiers define what the assessment is. Event context defines when and how it occurred. These responsibilities must remain separate.

When event identity is embedded into identifiers or inferred from structure:

- Assessment identity becomes unstable
- Longitudinal analysis is fragmented
- Implementations become inconsistent across partners
- Downstream systems must reverse-engineer meaning

Event context must be modeled using the appropriate fields defined in the StudentAssessment record.

Assessment Provider Responsibility

The assessment provider is responsible for ensuring that event identity is modeled explicitly through context fields and not embedded in identifiers or naming conventions across implementations:

- Represent timing and attempt context using defined event fields
- Keep assessment identity separate from event context
- Ensure consistency in how event identity is modeled across implementations
- Align event modeling decisions with governance expectations

Prohibited Patterns

The following patterns are not allowed because they conflate identity and event context and require downstream systems to reconstruct or infer meaning:

- Encoding BOY, MOY, EOY, or time-based labels in AssessmentIdentifier
- Using school year as part of the assessment identity when structure is unchanged
- Encoding event timing in identifiers instead of context fields
- Using AssessmentPeriod as a substitute for event identity instead of context
- Relying on naming conventions to convey timing or attempt information

Modeling event identity through explicit context fields ensures that assessment identity remains stable while event context remains interpretable, enabling consistent longitudinal analysis without requiring downstream systems to reconstruct meaning.

7. Descriptor and Namespace Governance

Descriptor and namespace governance define how meaning is preserved within an Ed-Fi integration.

Descriptors are not just technical values. They carry semantic meaning that must remain consistent, interpretable, and traceable back to the source assessment. Namespaces define ownership of that meaning.

A native integration preserves vendor-reported meaning at ingestion and ensures that any interpretation, normalization, or cross-assessment comparison occurs through transparent, governed downstream processes.

Cross-vendor comparability is not achieved by rewriting vendor data at ingestion. It is achieved through consistent modeling, preserved semantics, and explicit mapping layers applied after ingestion.

Incorrect descriptor governance does not break data loading. It breaks meaning.

7.1 Vendor Namespace Rules

Native integrations must preserve vendor-native score names, reporting methods, and performance level semantics exactly as defined in the vendor score report.

A native integration does not alter, simplify, or normalize vendor meaning during ingestion.

This applies to all **assessment-specific** descriptors, including:

- Assessment reporting method descriptors (e.g., score names)
- Performance level descriptors (e.g., proficiency categories)
- Assessment category descriptors
- Assessment period descriptors

If interpretation or grouping is required for analytics, it must be implemented through:

- governed downstream mapping layers, or
- clearly defined metadata aligned to vendor-specific values

At ingestion, preservation of meaning is the priority.

This ensures:

- Full transparency of what the vendor reported
- Traceability back to the original score report
- Ability to support multiple analytic interpretations
- Elimination of hidden transformation logic

Assessment Provider Responsibility

The assessment provider is responsible for ensuring that vendor-specific descriptors, including score names and performance level semantics, are preserved exactly as defined and consistently applied across implementations:

- Publish descriptor values exactly as defined in the vendor score report
- Use a vendor-specific namespace for all assessment-owned descriptors
- Preserve performance level definitions without remapping or simplification
- Ensure consistency of descriptor values across implementations
- Align descriptor usage with governance expectations

Prohibited Patterns

The following patterns are not allowed because they alter source meaning and require downstream systems to reconstruct or reinterpret vendor semantics:

- Normalizing or renaming vendor score descriptors at ingestion
- Mapping performance levels to simplified or local categories during ingestion
- Overwriting vendor descriptor values with implementation-specific definitions
- Collapsing distinct vendor measures into a single generic descriptor
- Embedding interpretation logic within descriptor values

Preserving vendor namespace semantics ensures that meaning remains intact at ingestion and that any analytical interpretation is transparent, governed, and reproducible.

7.2 Default Namespace Usage

Certain descriptors represent shared dimensions across the Ed-Fi data model and must remain in the default Ed-Fi namespace unless a defined extension applies.

These include:

- AcademicSubject
- GradeLevel
- Language

These descriptors are not owned by the assessment provider, even when referenced within assessment metadata.

Maintaining these descriptors in the default namespace ensures:

- Consistency across domains such as assessments, courses, and student records
- Reliable cross-domain analysis
- Elimination of conflicting definitions across implementations

Implementations may apply environment-specific mappings for shared cross-domain descriptors through governed configuration layers when required to align with local Ed-Fi environments. However, assessment-specific descriptors, including vendor-defined score names and performance levels, must remain vendor-defined at ingestion and must not be locally overridden during load processing.

Assessment Provider Responsibility

The assessment provider is responsible for ensuring that shared descriptors are correctly referenced from the default namespace and consistently applied across implementations:

- Use default Ed-Fi descriptors for shared dimensions
- Align descriptor values to those defined in the target implementation
- Avoid redefining shared descriptors within the vendor namespace
- Ensure consistent usage across integrations

Prohibited Patterns

The following patterns are not allowed because they introduce inconsistency across domains and require downstream systems to reconcile conflicting definitions:

- Defining AcademicSubject within a vendor namespace
- Defining GradeLevel within a vendor namespace
- Creating duplicate descriptor sets for shared dimensions
- Using inconsistent descriptor values across implementations

Using default namespace descriptors ensures cross-domain consistency and supports reliable integration across systems.

7.3 Descriptor Governance by Domain

Descriptor governance must clearly distinguish between assessment-specific descriptors and shared descriptors used across domains.

Assessment-specific descriptors are owned by the assessment provider and must be defined and maintained within the vendor namespace. Assessment-specific descriptors must not be locally overridden or remapped at ingestion because doing so compromises the semantic integrity of the assessment results.

These include:

- Assessment reporting method descriptors
- Performance level descriptors
- Assessment category descriptors
- Assessment period descriptors

Performance level semantics must be preserved as vendor-defined descriptor values. They must not be remapped to local or simplified categories at ingestion.

These descriptors are intrinsic to the assessment and must reflect vendor-defined meaning without modification.

In contrast, non-assessment descriptors:

- AcademicSubject
- GradeLevel
- Language

are shared across domains and must align to the implementation environment.

Failure to separate these responsibilities introduces ambiguity and breaks interoperability.

Assessment Provider Responsibility

The assessment provider is responsible for ensuring that descriptor ownership is clearly defined and consistently applied across implementations:

- Define assessment-specific descriptors within the vendor namespace
- Preserve vendor-defined meaning for all assessment descriptors
- Reference shared descriptors from the default namespace
- Avoid redefining or overriding shared descriptors
- Maintain consistent descriptor governance across integrations

Prohibited Patterns

The following patterns are not allowed because they blur ownership boundaries and require downstream systems to infer or reconcile descriptor meaning:

- Mixing assessment and non-assessment descriptors within the same namespace
- Redefining shared descriptors within the vendor namespace
- Using vendor namespaces for cross-domain descriptors
- Applying inconsistent governance rules across descriptor types

Clear separation of descriptor governance ensures that meaning remains consistent, ownership is explicit, and cross-domain integration remains reliable.

Part IV – Student Identity and Rostering

8. Student Identity Resolution

Student identity and rostering define how assessment results are aligned to the correct student within the Ed-Fi ecosystem.

This section establishes the requirements for ensuring that every StudentAssessment record resolves to a valid student and can be used for reporting, analysis, and longitudinal tracking.

Unlike data modeling, which determines how results are structured, identity and rostering determine whether those results can be used at all.
A dataset that does not resolve to a valid student identity may load partially or fail entirely, but in either case, it cannot support reliable analytics.

A native Ed-Fi assessment integration must ensure that:

- Student identity is explicitly defined and consistently applied
- Rostering is aligned to the system of record
- Identity resolution is deterministic, transparent, and repeatable

8.1 Why Identity Is the Foundation

All assessment results depend on a single requirement: each record must resolve to a valid *StudentUniqueId* in the Ed-Fi ODS.

This is not simply an API constraint. It is the mechanism that enables:

- Enrollment-aligned reporting
- Longitudinal analysis
- Cross-domain integration

When identity is not correctly resolved:

- StudentAssessment records are rejected or excluded
- Results cannot be attributed to the correct student
- Longitudinal data becomes fragmented or unusable

In practice, the majority of assessment integration challenges are not caused by score modeling issues. They are caused by student identity mismatches across systems. A native integration must treat identity resolution as a first-class requirement, not a downstream correction step.

8.2 Rostering Strategy

Rostering defines how students included in assessment data are aligned to the Ed-Fi environment, where assessment results are submitted.

The critical requirement is not the rostering platform itself. The requirement is that student identity alignment is explicit, consistent, and verifiable across systems.

Assessment integrations depend on accurate identity resolution. If rostered students cannot reliably align to the Ed-Fi StudentUniqueId values used by the receiving environment, assessment results become difficult to reconcile, longitudinal analysis becomes fragmented, and downstream systems must compensate for identity inconsistencies through custom logic and manual intervention.

Core Principle

Assessment vendors should roster from the same environment that receives assessment results whenever possible.

This ensures that:

- Student identifiers align to Ed-Fi StudentUniqueId
- Enrollment and assessment participation context remain consistent
- Identity mismatches are minimized
- Assessment results can be resolved directly to the Ed-Fi student record

Rostering is the operational bridge between vendor systems and the Ed-Fi identity model.

Recommended Implementation Model

A native integration typically follows a bidirectional pattern:

- The vendor retrieves roster and identity data from Ed-Fi
- The vendor submits assessment results back to Ed-Fi

This model establishes Ed-Fi as the system of record for both identity and assessment outcomes, reducing reconciliation complexity and strengthening longitudinal integrity across systems.

While this represents the strongest alignment pattern, implementations across the ecosystem vary in maturity and operational readiness. Multiple rostering approaches are currently used successfully across states and districts. These approaches are not operationally identical, but each can support a compliant integration when identity alignment is managed appropriately.

Rostering Approaches and Identity Alignment

The following rostering approaches represent common implementation patterns across the Ed-Fi ecosystem.

8.2.1 Native OneRoster API from Ed-Fi

An emerging implementation pattern is the exposure of OneRoster endpoints directly from the Ed-Fi API environment.

Beginning with Ed-Fi API v7.3.2, the Ed-Fi Alliance introduced support for native OneRoster API endpoints. This creates a standardized rostering model that aligns directly to the Ed-Fi system of record while leveraging a roster specification already familiar to many assessment vendors.

This approach provides several advantages:

- Ed-Fi remains the authoritative source for student identity and roster data
- Rostering aligns directly to StudentUniqueId
- Rostering workflows can operate through standardized APIs
- Intermediate transformation and synchronization layers are minimized

This pattern represents a strong, long-term interoperability model because it combines Ed-Fi's identity and integrity with OneRoster's operational familiarity.

At the same time, implementation maturity varies across the ecosystem, and not all current Ed-Fi environments expose or operationalize these endpoints consistently.

8.2.2 StudentAssessmentRegistration via Ed-Fi API

When native OneRoster endpoints are not available, vendors may retrieve roster and assessment participation data directly through Ed-Fi APIs using the StudentAssessmentRegistration resource.

The StudentAssessmentRegistration endpoint is supported in the Ed-Fi ODS/API v7.x series and is specifically designed to support assessment rostering and coordination workflows between education agencies and assessment vendors.

This resource provides:

- The set of students expected to participate in an assessment
- Associations between students and assessment administrations

- Context needed to align assessment results to the correct assessment event

Using StudentAssessmentRegistration helps ensure that:

- Ed-Fi remains the system of record
- Student identity aligns directly to StudentUniqueId
- Assessment participation is explicitly defined and governed

This approach supports the automation of assessment registration workflows that have historically depended on manual processes and spreadsheet-based coordination.

Operational adoption varies because not all Ed-Fi implementations currently operationalize these endpoints consistently. In those environments, additional coordination with student and enrollment resources may still be required.

8.2.3 OneRoster CSV Exports Derived from Ed-Fi

Some implementations generate OneRoster-compliant CSV exports directly from the Ed-Fi environment.

This approach preserves alignment to the Ed-Fi system of record while supporting vendors that already consume standard OneRoster CSV workflows.

This pattern:

- Preserves Ed-Fi as the authoritative source
- Supports batch-based rostering workflows
- Enables large-scale operational exchanges
- Provides a practical interoperability bridge for implementations not yet supporting native roster APIs

This model is commonly used in exchange-based implementations where standardized roster exports are generated centrally and distributed to vendors.

While this approach introduces more operational coordination than API-based patterns, it remains strongly aligned to Ed-Fi identity because the roster source originates from the Ed-Fi environment itself

8.2.4 External Rostering Systems

External rostering platforms such as ClassLink, Clever, and SIS-driven rostering workflows may also support successful assessment integrations when identity alignment is managed appropriately.

Some implementations successfully leverage external rostering providers in coordination with Ed-Fi environments, including models where roster synchronization and identity reconciliation processes are already operationally established.

These approaches can:

- Accelerate onboarding
- Reduce implementation friction
- Leverage existing district rostering investments
- Simplify operational coordination for vendors already integrated with these platforms

At the same time, these systems are not typically the Ed-Fi system of record. As a result, additional identity reconciliation and governance controls are often required to ensure accurate alignment with StudentUniqueId.

When rostering does not originate directly from the Ed-Fi environment, vendors assume additional responsibility for:

- Deterministic identity matching
- Cross-system reconciliation transparency
- Match rate monitoring
- Referential integrity validation between rostered students and Ed-Fi identity records

The success of these implementations depends on whether student identity alignment remains consistent, auditable, and operationally manageable over time.

Assessment Provider Responsibility

The assessment provider is responsible for ensuring that the selected rostering approach consistently supports accurate identity resolution and referential integrity across implementations.

This includes:

- Documenting the rostering approach used for each implementation
- Preserving alignment to StudentUniqueId
- Supporting deterministic and auditable identity matching
- Monitoring and reporting identity match rates
- Ensuring unresolved identity conflicts are surfaced and corrected
- Applying identity reconciliation processes consistently across implementations
- Ensuring rostering workflows remain operationally maintainable over time

Prohibited Patterns

The following patterns are not allowed because they weaken identity integrity and require downstream systems to reconstruct or infer student identity:

- Using roster data that cannot be deterministically aligned to StudentUniqueId
- Allowing identity matching behavior to vary unpredictably across implementations
- Treating identity reconciliation as a downstream responsibility
- Silently dropping unmatched students or unresolved records
- Using rostering workflows that cannot be audited or explained operationally
- Relying on manual or undocumented identity repair processes as part of normal operations

Rostering is not defined by the specific tool or platform being used. It is defined by whether student identity can be consistently aligned to the Ed-Fi system, where assessment results are stored.

The closer the rostering source is to the Ed-Fi system of record, the stronger the referential integrity, the lower the reconciliation burden, and the more reliable the integration becomes over time.

8.3 Student Identifier Selection and Configuration

Student identity must resolve to StudentUniqueId, which is the canonical identifier in Ed-Fi.

Rostering systems often provide multiple identifiers, including:

- District student IDs
- State student IDs
- SIS identifiers
- Platform-specific identifiers

A native integration must support explicit configuration of which identifier is used for matching.

This configuration must be:

- Visible
- Testable
- Consistently applied

The integration must not assume that any given identifier will automatically align with StudentUniqueId. Identifiers such as names or loosely defined attributes must not be used as primary matching keys.

Assessment Provider Responsibility

The assessment provider is responsible for ensuring that identity matching is configurable, deterministic, and consistently applied across implementations:

- Provide configurable identifier selection
- Support multiple identifier types
- Ensure matching resolves to StudentUniqueId
- Prevent use of non-deterministic identifiers
- Maintain consistent matching logic across implementations

Prohibited Patterns

The following patterns are not allowed because they introduce ambiguity in identity and require downstream systems to reconstruct or infer matches:

- Using names or non-unique attributes as primary identifiers
- Relying on implicit or undocumented matching logic
- Hardcoding identifier selection per implementation
- Using identifiers that cannot be mapped to StudentUniqueId

Using explicit, governed identifier configuration ensures that identity resolution is reliable, auditable, and consistent across systems.

8.4 Identity Mapping and Crosswalk

In many implementations, vendor identifiers do not directly match StudentUniqueId.

In these cases, identity must be resolved through a crosswalk process.

Identity mapping is not a one-time setup. It is an operational process that must support:

- Initial data loads
- Incremental updates
- Backfills and corrections

A valid identity mapping process includes:

- Deterministic matching logic
- Measurable match rates
- Repeatable execution
- Full auditability

This ensures that identity resolution is transparent and governed rather than implicit or hidden.

Assessment Provider Responsibility

The assessment provider is responsible for ensuring that identity mapping is implemented as a governed, repeatable process:

- Support crosswalk-based identity resolution
- Provide deterministic matching logic
- Enable match rate measurement and validation
- Ensure auditability of all identity mappings
- Support reprocessing of corrected records

Prohibited Patterns

The following patterns are not allowed because they obscure identity resolution and create untraceable data inconsistencies:

- Performing identity matching without auditability
- Using non-deterministic or heuristic-only matching
- Treating identity mapping as a one-time configuration
- Failing to measure or report match quality

Treating identity mapping as a governed process ensures that identity remains reliable, transparent, and correctable over time.

8.5 Referential Integrity and Load Enforcement

The Ed-Fi API enforces referential integrity.

A StudentAssessment record cannot be loaded unless:

- The referenced student exists
- The student is identified by a valid StudentUniqueId

This means identity resolution must occur before data submission, not after.

If identity is not resolved:

- Payloads are rejected
- Records are excluded
- Data completeness is compromised

Assessment Provider Responsibility

The assessment provider is responsible for ensuring that identity is fully resolved prior to submission and that all records reference valid students:

- Validate identity before loading

- Ensure all records resolve to StudentUniqueId
- Prevent partial or inconsistent submissions
- Maintain referential integrity across all data

Prohibited Patterns

The following patterns are not allowed because they break referential integrity and result in incomplete or unreliable data:

- Submitting records with unresolved student identity
- Allowing partial loads with missing student references
- Attempting to resolve identity after ingestion
- Ignoring or suppressing identity-related errors

Enforcing referential integrity ensures that all assessment data is complete, valid, and usable at the point of ingestion.

8.6 Handling Identity Failures

Identity failures must be explicitly surfaced and resolved. Unmatched records must never be silently dropped. A successful integration is not defined by what loads. It is defined by whether all data is accounted for.

A native integration must include staging for unmatched records, reconciliation reporting, clear error diagnostics, and reprocessing workflows after correction.

Assessment Provider Responsibility

The assessment provider is responsible for ensuring that identity failures are visible, traceable, and correctable:

- Surface unmatched records
- Provide clear failure diagnostics
- Support correction and reprocessing
- Maintain visibility into data completeness

Prohibited Patterns

The following patterns are not allowed because they hide data loss and undermine data integrity:

- Dropping unmatched records silently
- Reporting successful loads that exclude data
- Failing to provide diagnostics for identity failures
- Preventing reprocessing after correction

Handling identity failures transparently ensures that data completeness and quality are maintained.

8.7 District Visibility and Correction

Identity resolution must be visible and actionable by districts and implementation teams.

District users must be able to:

- View unmatched or incorrect records

- Understand why matching failed
- Provide corrections
- Re-run processing

Identity resolution cannot depend solely on vendor intervention.

Assessment Provider Responsibility

The assessment provider is responsible for enabling district-level visibility and correction of identity issues:

- Provide access to identity mismatch data
- Enable correction workflows
- Ensure transparency in matching logic
- Support operational resolution without vendor dependency

Prohibited Patterns

The following patterns are not allowed because they create dependency and prevent timely resolution:

- Hiding identity logic from users
- Requiring vendor intervention for corrections
- Preventing visibility into unmatched records
- Obscuring how identifiers are used for matching

Providing district visibility ensures that identity resolution becomes a governed operational process rather than a bottleneck.

8.8 Impact on Local Use and Longitudinal Integrity

Accurate student identity resolution is what makes assessment data usable beyond basic reporting.

When identity is aligned:

- Results align to enrollment at the time of assessment
- Growth analysis is accurate across time
- Cohort and subgroup analysis are reliable
- Cross-domain integration becomes possible

When identity is not aligned:

- Students appear duplicated or missing
- Longitudinal data is fragmented
- Reporting becomes unreliable
- Data cannot be trusted

Identity resolution is not a supporting function. It is the condition that determines whether an integration is usable at all. For this reason, student identity validation and rostering are required components of a native integration. These core requirements determine whether the integration is viable at all.

Part V — Integration Architecture and Operations

This section defines how an assessment integration moves from "a payload that can be loaded" to a true native integration that districts can operationalize safely at scale. The focus here is transport and execution discipline: how data is transformed, validated, transmitted, secured, reprocessed, and monitored without introducing fragility, tenant risk, or silent duplication.

Architecture is where good modeling either survives the real world or dies in production.

A vendor can implement perfect Ed-Fi modeling on paper and still deliver an integration that fails districts if it depends on direct database writes, cannot be re-run safely, mixes tenants, or collapses under real volumes during peak testing windows. The Native Integration Foundational Design Principles apply here, too. The integration must be operationally usable, governance-aligned, and stable across implementations. That requires explicit enforcement mechanisms, not hopeful assumptions.

9. Architecture Patterns

The patterns below are two common architectures that can meet assessment integration. They are not equal in effort, and each can be implemented well if it meets the technical requirements and the enforcement rules outlined later in this section.

9.1 Pattern A: Bidirectional Native API

In this pattern, the vendor owns the full integration runtime. The vendor maps and transforms assessment data from its internal systems into Ed-Fi resources and interacts directly with the district or state Ed-Fi ODS/API using REST. A native integration follows a bidirectional pattern:

- The vendor pulls roster data from Ed-Fi
- The vendor pushes assessment results to Ed-Fi

This bidirectional pattern is foundational to a compliant integration. Pulling roster data from Ed-Fi ensures that student identity, enrollment context, and assessment registration are aligned to the system of record where results will be stored. Pushing results back to Ed-Fi completes the integration by delivering outcomes that are already resolved to that same identity framework.

A native integration is defined not only by how data is written to Ed-Fi, but by how identity is sourced from it. If this pattern is not followed, identity mismatches become unavoidable. Vendors that rely on external rostering sources or delayed reconciliation introduce inconsistencies that break referential integrity, fragment longitudinal data, and increase operational burden on districts. Bidirectionality is not a convenience. It is the mechanism that ensures assessment data is accurate at load time and usable over time.

What it looks like in practice:

- The vendor sources the assessment data directly from its internal systems (such as application databases or service layers), rather than relying on external extracts.
- The vendor retrieves roster, enrollment, and assessment registration data from the Ed-Fi API to establish identity alignment.
- The vendor applies mapping and transformation logic to convert vendor-native data structures into Ed-Fi resources (Assessment, ObjectiveAssessment, StudentAssessment, StudentObjectiveAssessment).

- The vendor uses Ed-Fi API credentials issued per district or tenant to POST and PUT resources to the ODS/API.
- The vendor is responsible for sequencing, error handling, retries, and safe reprocessing.

Why this pattern works when done correctly:

- It enforces alignment to the Ed-Fi system of record for both identity and results, eliminating the need for downstream reconciliation.
- It aligns with the Ed-Fi architectural intent that integrations interact with the ODS through the API layer, not through database writes.
- It creates a clean operational boundary: the vendor produces standards-aligned resources, and the Ed-Fi implementation is responsible for storage and access.
- It supports real-time or near-real-time ingestion during testing windows when districts want timely results.
- It enables scalable, repeatable integrations across multiple districts and states.

What can go wrong if it is implemented carelessly:

- If the vendor does not pull roster data from Ed-Fi and instead relies on external sources, identity mismatches occur, leading to misaligned student records and broken longitudinal analysis.
- If the vendor does not implement safe-to-retry behavior, reprocessing creates duplicate StudentAssessment events and corrupts reporting.
- If the vendor fails to respect dependency order, payloads are rejected or partially loaded, resulting in incomplete hierarchy or missing metadata.
- If the vendor does not enforce proper tenant isolation and credential scoping, data boundaries can break, creating significant security and privacy risks.
- If the vendor pushes data without pre-validation, districts become the validation environment and spend significant time triaging avoidable errors.

Minimum enforcement expectations:

- The API-only rule must be respected. No direct database writes, no “patching the ODS,” and no shadow tables that bypass the API contract.
- Bidirectional integration must be implemented. Rostering and identity must be sourced from Ed-Fi, not external systems, unless explicitly required as a fallback.
- Safe-to-retry behavior must be engineered from the start, not added after duplicates are discovered.
- Dependency order must be enforced to maintain referential integrity.
- Errors must be transparent and recoverable. Silent drops are not acceptable.
- Validation must occur prior to submission to prevent avoidable downstream failures.

9.2 Pattern B: Bundle-Based (Earthmover + Lightbeam)

In this pattern, transformation, validation, and loading are explicitly separated into distinct stages prior to API submission, using a repeatable “bundle” workflow. Earthmover is used to transform and shape data, and Lightbeam is used to validate and load data into the Ed-Fi ODS/API reliably. The key concept is that these stages are independently managed, observable, and reusable across implementations.

What it looks like in practice:

- Vendor exports results in a defined vendor format, or a district extracts results via vendor export tooling.
- Earthmover converts the vendor format into Ed-Fi-shaped records using predefined transformation logic (housed in Bundles), often producing staged outputs that map 1:1 to Ed-Fi resources.
- Validation occurs before loading, including schema checks, dependency checks, descriptor checks, and key consistency checks.
- Lightbeam submits resources to the ODS/API in dependency order with robust retry logic and error reporting.

Why this pattern works when done correctly:

- It creates reusable mapping patterns and repeatable pipelines that can be operationalized across districts and states, especially when vendors provide consistent input structures. Bundles can often be reused across states with minimal adaptation when the same source data format is available.
- It allows strict technical enforcement before submission, which reduces the "district as QA environment" problem.
- It supports governance because the mapping logic is visible, reviewable, and version-controlled.
- It is often the fastest path to consistency across multiple vendor integrations because the same enforcement scaffolding is reused.

What can go wrong if it is implemented carelessly:

- If the bundle does not include stable natural keys, reruns create duplicates or overwrite the wrong records.
- If the pipeline treats validation as optional, bad data gets loaded and becomes harder to unwind.
- If the transformation logic embeds district-specific hacks, the workflow becomes non-portable and fragile.
- If Lightbeam loads are not monitored with clear error outputs, districts lose trust and stop operationalizing the feed.

Minimum enforcement expectations:

- Transform → validate → send is mandatory. If validation is skipped, the architecture is not bundle-based; it is just a delayed failure.
- Mapping logic must be designed to be reusable across districts without rewriting the model each time.

10. Descriptor Mapping Infrastructure

Descriptor management is a critical component of a native Ed-Fi assessment integration. While earlier sections define how descriptors should be modeled and how vendor semantics must be preserved, this section defines how descriptor behavior must function in a real-world implementation.

A native integration does not end with correctly assigning descriptor namespaces and values. It must support the realities of multi-state, multi-district deployments, where descriptor availability, naming conventions, and governance expectations vary across environments.

If descriptor handling is rigid, hidden, or hard-coded, the integration becomes brittle, non-portable, and difficult to govern. If descriptor handling is overly permissive or opaque, meaning becomes inconsistent and analytics become unreliable.

A native integration must strike a balance and source meaning at ingestion while enabling controlled, visible, and configurable interpretation at the local level.

10.1 Preservation at Ingestion

At ingestion, descriptor values must reflect exactly what the assessment provider reported.

This includes:

- Assessment reporting method descriptors (score names)
- Performance level descriptors (proficiency categories)
- Assessment category descriptors
- Assessment period descriptors
- Any vendor-defined categorical outputs

These values must:

- Remain in the vendor namespace
- Retain their original code values
- Not be renamed, simplified, or normalized during ingestion

Preserving vendor semantics ensures that:

- All data remains traceable to the original score report
- Meaning is not altered before governance is applied
- Distinct vendor-defined concepts remain distinguishable

A native integration preserves meaning first. Interpretation happens later.

Assessment Provider Responsibility

The assessment provider is responsible for ensuring that vendor-defined descriptor values are preserved exactly as reported:

- Preserve vendor-native descriptor values at ingestion
- Use vendor namespaces for all assessment-specific descriptors
- Maintain alignment with the vendor score report
- Avoid any transformation that alters meaning during ingestion
- Ensure consistency of descriptor behavior across implementations

Prohibited Patterns

The following patterns are not allowed because they alter source meaning and require downstream systems to reconstruct interpretation:

- Renaming descriptor values during ingestion
- Mapping performance levels to local categories at load time
- Collapsing distinct descriptor values into generalized categories
- Overwriting vendor-defined descriptor values without traceability

Preserving descriptor values at ingestion ensures that meaning remains intact and that all downstream interpretation is explicit and governed.

10.2 Local Mapping and Override

For shared, non-assessment descriptors such as AcademicSubject, GradeLevel, and Language, the assessment provider must align to the descriptors defined within the Ed-Fi implementation.

While a native integration may include default mappings, these are not authoritative. The receiving Ed-Fi environment defines the canonical descriptor values for these domains.

This requires the ability to map descriptor values to match:

- The namespace used by the implementation
- The code values used in the Ed-Fi ODS

This alignment is essential because these descriptors are used across domains. Misalignment prevents accurate joins between assessment results and enrollment, course, and student data.

This requirement applies only to shared descriptors. Assessment-specific descriptors must remain in the vendor namespace and must not be overridden.

Assessment Provider Responsibility

The assessment provider is responsible for ensuring that descriptor mapping is configurable and aligned to the receiving environment:

- Support environment-specific descriptor configuration
- Align shared descriptors to the target Ed-Fi implementation
- Preserve assessment-specific descriptors in the vendor namespace
- Document mapping rules and configuration behavior
- Ensure mappings are repeatable and version-controlled

Prohibited Patterns

The following patterns are not allowed because they create inconsistency and break interoperability:

- Assuming default descriptor values apply across all environments
- Redefining shared descriptors in vendor namespaces
- Overriding assessment-specific descriptor values at ingestion
- Applying undocumented or inconsistent mappings

Proper descriptor alignment ensures that data integrates cleanly across domains without requiring downstream reconciliation.

10.3 No Hard-Coded Descriptor Logic

Descriptor handling must never be hard-coded into transformation logic.

Hard-coded descriptors create:

- Non-portable pipelines that break across implementations
- Hidden dependencies that are difficult to diagnose
- Barriers to reuse across districts and states

Instead, descriptor logic must be:

- Parameterized (e.g., namespace variables)
- Externalized into mapping layers or configuration files
- Adjustable without requiring code changes

This applies to all descriptor categories, including:

- Reporting methods
- Performance levels
- Assessment periods
- Result data types

Assessment Provider Responsibility

The assessment provider is responsible for ensuring that descriptor logic is configurable and externalized:

- Parameterize descriptor values and namespaces
- Maintain mappings in configuration artifacts
- Support updates without code changes
- Document descriptor handling logic
- Ensure portability across implementations

Prohibited Patterns

The following patterns are not allowed because they prevent scalability and governance:

- Hard-coding descriptor values in transformation logic
- Requiring code changes for descriptor updates
- Embedding descriptor logic in undocumented scripts
- Using hidden or implicit descriptor rules

Descriptor logic must be configurable to support scalable, multi-partner integrations.

10.4 Transparency and Traceability

Descriptor mappings must be visible, auditable, and explainable.

A native integration must make it possible to answer:

- What descriptor value was sent?
- What was the original vendor value?
- Was a mapping applied?
- What rule produced the mapped value?

To support this, descriptor handling must include:

- Explicit mapping tables or configuration artifacts
- Clear documentation of namespace usage and mapping rules
- Version tracking of descriptor mappings over time

Mappings must not be hidden in code or dependent on institutional knowledge.

Assessment Provider Responsibility

The assessment provider is responsible for ensuring full transparency of descriptor behavior:

- Maintain visible mapping tables
- Document mapping rules and namespace usage
- Track mapping versions over time
- Preserve original vendor values where applicable
- Enable audit and troubleshooting without code inspection

Prohibited Patterns

The following patterns are not allowed because they obscure behavior and prevent governance:

- Hiding mappings in transformation code
- Applying undocumented transformations
- Requiring code inspection to understand descriptor behavior
- Losing original vendor values after mapping

Transparency ensures that descriptor behavior is governable, auditable, and trustworthy.

10.5 Multi-Partner Scalability

Descriptor mapping must support reuse across multiple implementations.

A native integration should be able to:

- Deploy the same transformation logic across districts and states
- Apply different descriptor mappings per environment
- Onboard new partners without rewriting pipelines

This requires:

- Separation of mapping logic from transformation logic
- Environment-specific configuration layers
- Consistent input structures

When implemented correctly, this enables:

- Faster onboarding
- Lower implementation cost
- Consistent governance

Assessment Provider Responsibility

The assessment provider is responsible for ensuring that descriptor mapping scales across partners:

- Reuse transformation logic across implementations
- Support environment-specific configuration
- Separate configuration from code
- Maintain consistent data structures
- Avoid per-partner customization

Prohibited Patterns

The following patterns are not allowed because they prevent scalability:

- Rewriting descriptor logic per district
- Embedding partner-specific mappings in core logic
- Maintaining separate pipelines without governance
- Treating descriptor mapping as a one-off process

Scalable descriptor mapping ensures consistent behavior across all implementations.

10.6 Change Control

Descriptor mappings evolve over time as:

- Vendors introduce new values
- States update standards
- Districts refine reporting definitions

A native integration must support:

- Versioning of descriptor mappings
- Controlled updates with documentation
- Traceability of mapping history

Changes must be:

- Reviewed through governance processes
- Tested before deployment
- Documented with clear downstream impact

Without change control, descriptor drift breaks comparability and trust.

Assessment Provider Responsibility

The assessment provider is responsible for ensuring that descriptor changes are governed and traceable:

- Version descriptor mappings
- Document all changes
- Test updates before deployment
- Maintain mapping history
- Align changes with governance processes

Prohibited Patterns

The following patterns are not allowed because they create inconsistency and loss of trust:

- Changing mappings without documentation
- Applying updates without testing
- Losing historical mapping context
- Implementing uncontrolled overrides

Controlled change management ensures that descriptor meaning remains stable and comparable over time.

11. API Interaction and Safe Reprocessing

API interaction and safe reprocessing are the enforcement backbone of a native Ed-Fi assessment integration. These requirements ensure that integrations are not only technically valid but also operationally reliable, repeatable, and safe to run in production environments.

A native integration must treat the Ed-Fi ODS/API as an API-driven system. It must respect dependency order, enforce validation, and support deterministic reprocessing.

If these requirements are not met, integrations may:

- Load successfully once but fail under reprocessing
- Create duplicate records or corrupt longitudinal data
- Require manual intervention to repair data integrity

A production-ready integration is not defined by whether it can load data once.

It is defined by whether it can be executed repeatedly without corruption.

11.1 API Interaction Rules

The Ed-Fi ODS/API must be treated as an API-driven system. All interactions must occur through the REST API layer.

The API is where Ed-Fi enforces:

- Validation rules
- Referential integrity
- Descriptor constraints
- Authorization and access control

Bypassing the API bypasses governance.

Core rules:

- No direct database writes. Ever.
- No temporary database patches to fix vendor modeling
- No bypass mechanisms that write directly to ODS tables

Direct database interaction:

- Breaks validation and consistency rules
- Creates incompatibility across ODS versions
- Removes the ability for districts to govern and troubleshoot using standard tools

Assessment Provider Responsibility

The assessment provider is responsible for ensuring that all integration behavior respects the API contract:

- Submit all data through approved Ed-Fi API endpoints
- Respect validation and authorization rules
- Preserve API error responses for troubleshooting
- Ensure compatibility across Ed-Fi API versions

- Avoid any mechanism that bypasses API enforcement

Prohibited Patterns

The following patterns are not allowed because they bypass governance and introduce instability:

- Direct database writes into the ODS
- Manual table patches to fix integration errors
- Shadow write mechanisms outside the API
- Ignoring or suppressing API validation errors
- Treating API validation as optional

API-only interaction ensures that integrations remain secure, governable, and consistent across implementations.

11.2 Dependency Order

The Ed-Fi ODS/API cannot accept records that reference metadata that does not exist.

A native integration must load data in the correct dependency order.

Required load order:

- Assessment
- ObjectiveAssessment
- StudentAssessment (including nested StudentObjectiveAssessment results)

This order reflects how the data model is constructed:

- Assessment defines the overall instrument
- ObjectiveAssessment defines the structure and hierarchy
- StudentAssessment defines the student event
- StudentObjectiveAssessment represents detailed results within that event

StudentObjectiveAssessment is not a separate entity. It is a collection within StudentAssessment and must align to both:

- The StudentAssessment event
- The ObjectiveAssessment hierarchy

Operational Implications

- Assessment and ObjectiveAssessment must exist before student results are submitted
- Missing or incorrect hierarchy can be resolved before loading results
- Descriptor or metadata issues can be corrected at the metadata level
- StudentAssessment payloads must align with the pre-existing structure

A practical enforcement pattern:

- Stage and validate Assessment and ObjectiveAssessment
- Load metadata into the ODS
- Then construct and submit StudentAssessment payloads

Assessment Provider Responsibility

The assessment provider is responsible for enforcing dependency order and ensuring recoverable load behavior:

- Load Assessment before dependent records
- Load ObjectiveAssessment before student results
- Ensure StudentAssessment references valid metadata
- Ensure nested StudentObjectiveAssessment aligns to hierarchy
- Detect and handle dependency failures before continuing

Prohibited Patterns

The following patterns are not allowed because they create incomplete or misleading data states:

- Loading student results before assessment metadata
- Loading objective-level results before hierarchy definitions
- Continuing loads after dependency failures
- Treating partial loads as successful
- Repairing dependency issues manually outside the pipeline

Enforcing dependency order ensures that structure exists before results, preserving data integrity and enabling reliable troubleshooting.

11.3 Safe Reprocessing and Duplicate Prevention

Safe reprocessing is the difference between an integration that can be operated in production and one that corrupts the dataset with every rerun.

A native integration must be engineered so that it can be executed repeatedly without:

- Creating duplicate records
- Corrupting longitudinal history
- Overwriting unrelated events

Reprocessing scenarios are not edge cases. They are routine and must be supported:

- Historical backfills
- Vendor score corrections
- Vendor resends
- District-requested reloads
- Environment rebuilds
- ODS upgrades

If an integration cannot safely handle these scenarios, it is not production-ready.

Core Requirements

Stable natural keys must exist for all resources

- The integration must behave deterministically
- The same input must always produce the same resource identity
- Reprocessing must not create duplicate StudentAssessment events
- Reprocessing must not overwrite non-equivalent events

What Stable Keys Mean in Practice

Assessment

- Keys remain stable across years unless the assessment materially changes

ObjectiveAssessment

- Keys remain stable and aligned to the hierarchy

StudentAssessment

- Keys must represent a unique student attempt event
- Must include event identity (e.g., AdministrationDate, SchoolYear)

StudentObjectiveAssessment

- Keys must align to both:
  - The StudentAssessment event
  - The ObjectiveAssessment definition
- Must not rely on parsing score names

Reprocessing Behavior Expectations

- Reruns must upsert existing records, not create new ones
- Distinct attempts must generate distinct keys
- Corrections must update the existing record for that event
- Reprocessing must not depend on delete-and-reload strategies

Why AdministrationDate Is Critical

AdministrationDate is required for safe reprocessing.

Without it:

- Multiple attempts cannot be reliably distinguished
- Reloads may create duplicate StudentAssessment records
- Corrections cannot safely replace prior results
- Longitudinal analysis becomes unreliable

When AdministrationDate is missing, reprocessing behavior becomes guesswork—and guesswork is not acceptable in a native integration.

Assessment Provider Responsibility

The assessment provider is responsible for ensuring that reprocessing is safe, deterministic, and governed:

- Define stable natural keys for all resources
- Ensure deterministic behavior across runs
- Support updates for corrected records
- Distinguish separate attempts using event context
- Prevent duplicate StudentAssessment events
- Avoid destructive reload strategies

- Validate reprocessing behavior before production

Prohibited Patterns

The following patterns are not allowed because they create data corruption and unreliable longitudinal behavior:

- Creating new StudentAssessment records on every rerun
- Keying StudentAssessment only by student and assessment
- Using delete-and-reload as the primary recovery strategy
- Overwriting non-equivalent attempts
- Omitting AdministrationDate when needed
- Using batch IDs or runtime artifacts as stable keys

Safe reprocessing ensures that integrations can handle real-world operational scenarios without corrupting data, enabling reliable longitudinal analysis and repeatable execution.

12. Security and Tenant Isolation

Security is not an add-on. In assessment integrations, security failures often become privacy failures, and tenant isolation failures become data exposure incidents.

A native integration must demonstrate strong data separation, appropriate access controls, and auditable behavior across all environments. Security must be designed into the integration architecture from the beginning, not layered on after implementation.

This section defines the minimum expectations for credential management, access control, auditability, and tenant isolation required for a production-ready integration.

12.1 District-Scoped Credentials

Every district or tenant must have its own API credentials. Credentials must not be shared across districts, even when the assessment provider and integration logic are the same.

District-scoped credentials enforce tenant isolation at the access layer and ensure that integration activity can be clearly attributed to a specific district and integration client.

This requirement ensures that:

- Tenant boundaries are enforced through authentication
- The impact of a credential compromise is limited to a single tenant
- Audit trails can accurately trace actions to a specific district

Assessment Provider Responsibility

The assessment provider is responsible for ensuring that credentials are properly scoped and managed across all implementations:

- Issue separate credentials for each district or tenant
- Use separate credentials for production and non-production environments
- Prevent credential reuse across tenants
- Maintain clear ownership and tracking of credential usage
- Support credential revocation and re-issuance

Prohibited Patterns

The following patterns are not allowed because they break tenant isolation and increase security risk:

- Sharing credentials across multiple districts
- Using the same credentials for production and non-production environments
- Maintaining credentials without ownership or tracking
- Reusing credentials after tenant scope changes
- Continuing to use credentials after revocation or suspected compromise

District-scoped credentials ensure that access boundaries are enforced and that integration activity remains traceable and governable.

12.2 Least-Privilege Claimsets

API credentials must use claimsets that grant only the permissions required for the integration.

The integration's technical access scope must align with its intended function. If the integration only needs to write assessment data and read limited supporting data, it must not have permissions to modify unrelated domains.

Limiting permissions reduces risk and ensures that access is intentional and controlled.

This ensures that:

- Accidental writes to unrelated domains are prevented
- The impact of compromised credentials is limited
- Integration behavior aligns with governance expectations

Assessment Provider Responsibility

The assessment provider is responsible for ensuring that claimsets are restricted to the minimum required permissions:

- Request only the permissions needed for the integration
- Avoid granting access to unrelated domains
- Align claimsets to the integration's functional scope
- Review and validate permissions before production deployment
- Update permissions when integration scope changes

Prohibited Patterns

The following patterns are not allowed because they create unnecessary access risk:

- Granting broad or administrative claimsets
- Providing permissions to modify unrelated Ed-Fi domains
- Reusing elevated credentials for integration runtime
- Expanding permissions to compensate for design gaps
- Leaving unused or outdated permissions in place

Least-privilege claimsets ensure that integrations operate within clearly defined and controlled boundaries.

12.3 Audit, Rotation, and Tenant Model

A native integration must support auditability, credential rotation, and strong tenant isolation across all components of the system.

Audit Logging

Audit logs must capture:

- What data was sent
- When it was sent
- To which district/tenant
- Which credential identity was used
- What the API returned, including errors

Audit logs must support:

- Troubleshooting by vendors and implementation teams
- District-level visibility into integration behavior
- Governance review and compliance validation

Without audit logs, it is not possible to determine whether data was never sent, rejected, or incorrectly processed.

Credential Rotation

Credentials must be rotatable without breaking the integration.

Rotation must be:

- Routine, not emergency-only
- Supported without code changes
- Executable without downtime

The integration must also support:

- Immediate revocation if compromise is suspected
- Controlled re-issuance of credentials
- Separation of production and non-production credentials

Assessment data must not be copied into lower environments without masking or explicit authorization.

Strong Tenant Model

Tenant isolation must be enforced across all layers of the integration:

- Credential scoping
- Data processing pipelines
- Storage layers
- Logging and observability systems

This is especially critical for architectures that include shared infrastructure or intermediate data stores.

A strong tenant model ensures that:

- Data is never exposed across districts
- Processing remains isolated per tenant
- Observability and logging do not leak cross-tenant information

Assessment Provider Responsibility

The assessment provider is responsible for ensuring that security operations are auditable, maintainable, and tenant-isolated:

- Maintain comprehensive audit logs for all API interactions
- Support credential rotation without requiring code changes
- Support immediate credential revocation and re-issuance
- Enforce separation between production and non-production environments
- Ensure tenant isolation across processing, storage, and logging
- Prevent any cross-tenant data access or exposure

Prohibited Patterns

The following patterns are not allowed because they create security gaps and risk data exposure:

- Lack of audit logging for integration activity
- Credential rotation requiring code changes
- Sharing data between production and non-production environments without controls
- Logging that mixes or exposes data across tenants
- Cross-tenant access paths in processing or storage
- Inability to trace actions to a specific credential or tenant

Security and tenant isolation are required conditions for a native integration.

An integration that cannot enforce tenant boundaries, audit behavior, and manage credentials securely cannot be considered production-ready.

13. Scheduling, Monitoring, and Change Management

Scheduling, monitoring, and change management determine whether an integration is operationally sustainable or fragile.

A native integration must not only produce correct data—it must do so reliably under real-world conditions, including peak load, failures, corrections, and system changes.

An integration that requires manual intervention for routine operation is not production-ready.

This section defines the expectations for runtime design, observability, and change governance required for a scalable and trustworthy integration.

13.1 Batch Planning and Runtime Design

Assessment integrations are inherently bursty. Large volumes of data are generated during defined testing windows, including:

- Beginning-of-year assessments
- Interim benchmark cycles
- End-of-year summative testing

- District-wide retesting events

A native integration must be designed to handle both peak load conditions and full reprocessing scenarios.

Batch planning must account for:

- Peak record volumes
- Dependency ordering across resources
- Tenant isolation during concurrent district loads
- Full-year reload scenarios

The integration must explicitly support:

- Initial historical loads (large backfills)
- Incremental loads (daily or periodic updates)
- Correction loads (vendor resends)
- Full reprocessing events

A common failure pattern is designing only for incremental loads and failing under full reload conditions. A native integration must support both without degradation.

Retry Logic

Retry logic must be deterministic, bounded, and coordinated with safe reprocessing.

The integration must:

- Retry transient API failures
- Respect HTTP response codes
- Avoid infinite retry loops
- Log retry behavior clearly

Retries must not create duplicate StudentAssessment events and must respect dependency order. If a dependency fails, downstream processing must pause until resolved.

Backoff Strategy

During peak load periods, APIs may throttle or degrade.

The integration must:

- Respect API rate limits
- Use controlled or exponential backoff
- Avoid overwhelming the ODS/API during outages
- Prevent cascading failures across tenants

Backoff must work in coordination with retry logic so that failures are spaced, logged, and diagnosable.

Parallelization Safety

Parallelization improves throughput but introduces risk if not controlled.

Parallelization must:

- Respect tenant boundaries
- Respect dependency order
- Avoid race conditions across resources
- Prevent duplicate writes

Safe parallelization includes:

- Parallel processing by district tenant
- Parallel loading of StudentAssessment after metadata is established
- Bounded batch processing of StudentObjectiveAssessment

Unsafe patterns include:

- Loading dependent resources simultaneously without validation
- Running full reloads and incremental loads concurrently for the same tenant
- Multi-threaded writes without stable key enforcement

Parallelization must be tested at scale prior to production.

Assessment Provider Responsibility

The assessment provider is responsible for ensuring that runtime behavior is scalable, controlled, and resilient:

- Design for peak and full-load scenarios
- Implement deterministic retry logic
- Implement controlled backoff strategies
- Enforce dependency-aware execution
- Ensure safe parallelization across tenants
- Validate runtime performance under load conditions
- Prevent duplicate or conflicting writes

Prohibited Patterns

The following patterns are not allowed because they create instability and operational risk:

- Designing only for incremental loads
- Infinite or uncontrolled retry loops
- Ignoring API rate limits
- Parallel execution that violates dependency order
- Running conflicting load processes against the same tenant
- Retry strategies that create duplicate records

Well-designed runtime behavior ensures that integrations remain stable under load and recover gracefully from failure.

13.2 Monitoring and Observability

If you cannot see what the integration is doing, you cannot govern it.

Observability transforms an integration from a black box into accountable infrastructure. It enables troubleshooting, validation, and governance oversight.

Human-Readable Error Logs

Error logs must be:

- Human-readable
- Tenant-scoped
- Resource-specific
- Time-stamped
- District-auditable

Logs must clearly answer:

- What resource failed?
- Why did it fail?
- Was it a validation error?
- Was it a dependency issue?
- Was it a descriptor mismatch?
- Was it an authentication failure?

Opaque or technical-only logs are not acceptable. Districts must be able to interpret failures without vendor intervention.

Reconciliation Reporting

Reconciliation reporting ensures that intended data matches actual outcomes.

At a minimum, reporting must include:

- Count of records attempted
- Count of records successfully loaded
- Count of records rejected
- Breakdown by resource type
- Breakdown by error category

Advanced reconciliation should include:

- Student-level discrepancies
- Objective-level discrepancies
- Duplicate detection
- Missing dependency detection

Reconciliation is critical during:

- Initial go-live
- Full-year reloads
- Vendor correction resends
- ODS upgrades

Without reconciliation, districts are forced to trust integration outcomes without validation.

Resource-Level Error Breakdown

Errors must be categorized to enable rapid diagnosis and governance review.

Categories include:

- Assessment metadata failures
- ObjectiveAssessment structure failures
- StudentAssessment event identity failures
- StudentObjectiveAssessment hierarchy failures
- Descriptor resolution failures
- Referential integrity failures

Without structured categorization, large failure volumes become unmanageable.

Assessment Provider Responsibility

The assessment provider is responsible for ensuring that integration behavior is fully observable and auditable:

- Provide human-readable error logs
- Ensure logs are tenant-scoped and time-stamped
- Produce reconciliation reports for all runs
- Categorize errors at the resource level
- Enable district-level visibility into failures
- Support troubleshooting without requiring code access

Prohibited Patterns

The following patterns are not allowed because they prevent governance and troubleshooting:

- Opaque or unreadable logs
- Logs that require vendor interpretation
- Lack of reconciliation reporting
- Failure to categorize errors
- Missing visibility into failed records

Observability ensures that integration behavior can be understood, trusted, and governed.

13.3 Change Management and Versioning

Native integrations operate in evolving ecosystems. Without disciplined versioning and governance, change introduces silent data corruption and breaks longitudinal comparability.

There are three independent version layers that must be tracked and managed.

Assessment Version

This refers to changes in the assessment instrument itself.

Examples of changes include:

- Structural hierarchy changes
- Score calculation changes
- Performance level definition changes
- Statistical model recalibration
- Standards alignment changes

- Addition or removal of subtests or measures

These changes require governance decisions:

- Should the AssessmentIdentifier change?
- Does comparability across years remain valid?
- Should AssessmentFamily grouping change?

Failure to properly manage assessment versioning results in misleading longitudinal analysis.

Ed-Fi Data Standard Version

Ed-Fi Data Standard versions evolve and may introduce:

- New fields
- Deprecated fields
- Updated validation rules
- Descriptor constraints
- API behavior changes

The integration must:

- Explicitly track the target Ed-Fi version
- Be tested before upgrades
- Support migration planning

Silent incompatibility with new versions is not acceptable.

Integration (Bundle) Version

The transformation layer must be versioned independently.

This includes:

- Mapping logic updates
- Descriptor handling updates
- Hierarchy modeling changes
- Performance improvements
- Bug fixes

Versioning must include:

- Documentation
- Release notes
- Audit traceability
- Regression validation

Without versioning, districts cannot diagnose changes in data behavior.

Governance Triggers

The following events require formal governance review:

- Structural hierarchy changes
- Score method changes

- Performance level definition changes
- Statistical model changes
- Descriptor namespace changes
- Identifier construction changes
- Addition or removal of objective levels

Governance must evaluate the impact on:

- Longitudinal comparability
- Cross-district comparability
- Growth models
- Reporting pipelines
- Transcript reconstruction

No structural change should be deployed without governance review.

Assessment Provider Responsibility

The assessment provider is responsible for ensuring that all changes are controlled, documented, and governed:

- Track all version layers independently
- Document all changes and impacts
- Test changes prior to deployment
- Maintain version traceability
- Trigger governance review for structural changes
- Prevent silent or undocumented updates

Prohibited Patterns

The following patterns are not allowed because they introduce instability and loss of trust:

- Untracked or undocumented changes
- Silent updates to mappings or structure
- Deploying changes without testing
- Ignoring version dependencies
- Failing to assess longitudinal impact

Disciplined change management ensures that integrations remain stable, interpretable, and trustworthy over time.

Part VI — Validation and Certification

Testing is the enforcement layer of the playbook. An integration can produce technically valid Ed-Fi payloads and still fail the definition of a native integration if the data are ambiguous, incomplete for local use, or unstable under reprocessing.

The purpose of this validation framework is not to restate modeling rules, but to enforce them. Each validation category confirms that the requirements defined in Parts III–V are implemented correctly and consistently.

A native integration must be analytically usable without vendor-specific downstream logic. These validations ensure that the requirement is met before an integration is considered production-ready.

14. Validation Framework

Validation requirements are grouped into categories that reflect the core failure modes seen in assessment integrations. Each category defines what must be validated, what common failures look like, and what constitutes a passing result.

14.1 Structural Validation

Structural validation enforces the requirements defined in Part III (Assessment Definition and Identity, Hierarchy, and Results Placement).

It confirms that the dataset reflects the true structure of the assessment and can be interpreted without inference.

Structural validation must verify:

- Hierarchy integrity
    - Assessment exists at the level where overall results are defined
    - ObjectiveAssessment exists when subscores are present
    - ObjectiveAssessment is recursive when the vendor structure is multi-level
    - StudentAssessment and StudentObjectiveAssessment mirror that structure
- Subject resolution
    - Each Assessment resolves to a single AcademicSubject
    - Composite is used only when representing cross-subject outcomes
    - Subject meaning is not derived from score names
- Result grain alignment
    - Overall results are stored at StudentAssessment
    - Subcomponent results are stored in StudentObjectiveAssessment
    - No mixing of grains across levels

Common Failure Patterns

- Flat modeling with all scores at StudentAssessment
- Objectives defined, but no student objective results provided
- Multi-subject assessments requiring inference
- Composite results modeled at the wrong level

Passing Result

The hierarchy is complete, faithful to the vendor score report, and directly interpretable without vendor-specific logic.

14.2 Referential Integrity

Referential integrity enforces the dependency chain defined in Part III and Part V (API Interaction and Dependency Order).

It ensures that all relationships between entities are complete and navigable.

Validation must verify:

- Complete dependency chain:
  - Assessment → ObjectiveAssessment → StudentAssessment
- All references resolve:
  - StudentAssessment references a valid Assessment
  - StudentObjectiveAssessment aligns to a valid ObjectiveAssessment
- No orphaned records:
  - No StudentObjectiveAssessment without a matching structure
  - No hierarchy elements missing parents
- No “ghost definitions”:
  - No structure defined without corresponding results when expected

Common Failure Patterns

- Results submitted before the structure exists
- Broken references to missing objectives
- Inconsistent identifiers across runs
- Structure loaded separately from results

Passing Result

All data is fully connected and navigable from assessment definition to student outcomes with no breaks in the chain.

14.3 Descriptor Validation

Descriptor validation enforces the requirements defined in Part II (Descriptor Governance) and Section 10 (Descriptor Mapping Infrastructure).

It ensures that meaning is preserved and governed.

Validation must verify:

- Namespace alignment
  - Vendor descriptors remain in vendor namespace
  - Shared descriptors use Ed-Fi namespace
  - Namespace usage is consistent across environments
- Vendor semantics preserved
  - Performance levels are not remapped at ingestion
  - Descriptor meaning matches vendor definitions
- No reuse errors
  - Score descriptors used only for quantitative results
  - Performance descriptors used only for categorical interpretation
- No semantic flattening
  - Distinct values are not collapsed
  - Differences remain auditable

Common Failure Patterns

- Mixing score and performance descriptors

- Overwriting vendor semantics
- Reusing descriptors across categories
- Flattening multiple values into one

Passing Result

Descriptors retain their intended meaning, are consistently applied, and remain governable over time.

14.4 Event Completeness

Event completeness enforces the requirements defined in Section 6 (Event Identity and Longitudinal Integrity).

It ensures that every StudentAssessment represents a complete, interpretable event.

Validation must verify:

- Required fields:
  - SchoolYear
  - AdministrationDate
  - AssessmentPeriod (when applicable)
  - WhenAssessedGradeLevel
  - RetestIndicator (when applicable)
- Attempt distinguishability:
  - Multiple attempts do not collide
  - AdministrationDate differentiates events
- Enrollment alignment:
  - Results can be aligned to enrollment context
  - Off-cycle testing is handled correctly
- Longitudinal stability:
  - SchoolYear is consistent
  - Multi-year comparisons are valid

Common Failure Patterns

- Missing AdministrationDate
- Missing SchoolYear
- Period embedded in identifiers
- Missing grade context

Passing Result

Every StudentAssessment represents a clearly defined event that can be interpreted and used longitudinally without guesswork.

14.5 Student Identity Validation

Student identity validation enforces the requirements defined in Part IV (Student Identity and Rostering).

It ensures that results resolve to the correct student records.

Validation must verify:

- Match rates are measured and reported
- Unmatched records are surfaced and not silently dropped
- Identity resolution occurs before data load
- Referential integrity is maintained for all student records

Common Failure Patterns

- Unmatched records dropped silently
- Identity resolution deferred until after load
- Low match rates without visibility
- Inconsistent identifier usage

Passing Result

All student results resolve to valid Ed-Fi student records, with unmatched cases clearly surfaced and actionable.

14.6 Safe Reprocessing Simulation

Safe reprocessing validation enforces the requirements defined in Section 11 (Safe Reprocessing and Duplicate Prevention).

It confirms that the integration behaves deterministically under repeated execution.

Validation must verify:

- Stable natural keys across all resources
- Deterministic behavior across runs
- No duplicate creation during reruns
- Correct handling of updates vs new events

Required simulation scenarios:

- Repeat-run simulation
- Correction simulation
- Full-year reload simulation
- Backfill simulation

Common Failure Patterns

- Duplicate StudentAssessment creation
- Missing AdministrationDate causing collisions
- Keys dependent on runtime behavior
- Incorrect update vs insert behavior

Passing Result

Reprocessing is safe, deterministic, and does not corrupt data or create duplicates.

14.7 Dependency Order Validation

Dependency validation enforces the requirements defined in Section 11.2 (Dependency Order).

It ensures that sequencing and recovery behavior are correct.

Validation must verify:

- Correct load order:
  - Assessment
  - ObjectiveAssessment
  - StudentAssessment
- Runtime behavior:
  - API rejects out-of-order submissions
  - Errors are clearly logged
- Retry and recovery:
  - Dependencies retried correctly
  - No skipping forward
  - No duplicate creation
- Completeness:
  - No partial loads treated as success

Common Failure Patterns

- Loading results before metadata
- Silent failures in dependency chains
- Retry loops creating duplicates
- Partial loads considered complete

Passing Result

The integration enforces sequencing, handles failures predictably, and produces complete datasets.

14.8 Prohibited Pattern Detection

This validation enforces all prohibited patterns defined across Parts III–V.

Rather than restating them, this section defines how they are detected.

Validation must confirm the absence of:

- Multi-subject top-level assessments
- Composite results stored as objectives
- Time elements embedded in identifiers
- Missing event identity fields
- Pull-only architectures without a load path
- Pseudo-scores used for flags
- Collapsed hierarchy where structure exists

Detection Methods

Automated checks may include:

- Missing SchoolYear or AdministrationDate
- Identifier pattern detection (BOY/MOY/EOY)
- Score placement inconsistencies
- Descriptor misuse patterns

Manual review must validate:

- Structural correctness
- Semantic integrity
- Alignment with vendor score reports

Passing Result

No prohibited patterns are present, and the integration does not rely on downstream reconstruction of meaning.

15. Certification Checklist

Certification is the formal confirmation that an integration meets the Native Integration Foundational Design Principles across modeling, architecture, runtime behavior, and governance. Certification requires that integration behavior is transparent and explainable without reliance on transformation code inspection.

Certification is not a single test. It is a lifecycle validation that confirms the integration behaves correctly under real-world conditions.

This checklist is a verification instrument. No requirements are introduced here. Each item confirms that the requirements defined in Parts III–VI have been implemented and validated.

An integration must pass all applicable checks to be considered native, production-ready, and certifiable.

15.1 Structural Integrity

(Validates: Part III, Section 5, Section 10, Validation 14.1 & 14.3)

- Overall assessment-level results are stored at StudentAssessment
- Objective-level results (strands, domains, skills) are stored at StudentObjectiveAssessment
- Subscores are not stored at StudentAssessment
- Exactly one AcademicSubject exists at the Assessment level
- Hierarchy structure matches the vendor score report
- ObjectiveAssessment recursion is implemented where required
- Descriptor categories are not structurally misused
- Vendor-native score names are preserved
- Vendor-native performance level values are preserved at ingestion
- Composite subject is used correctly when representing cross-subject outcomes
- No multi-subject top-level assessments exist

15.2 Student Identity and Rostering

(Validates: Part IV, Validation 14.5)

- Student identifier source is explicitly configured and documented
- Identity mapping or crosswalk process is implemented
- Match rates are measured and reported
- All StudentAssessment records resolve to a valid StudentUniqueId prior to load
- Unmatched student records are surfaced and reported

- Unmatched records are not silently dropped
- District users can view identity mismatches
- District users can correct identity mismatches without vendor intervention

15.3 Event Completeness

(Validates: Section 6, Validation 14.4)

- SchoolYear is populated
- AdministrationDate is populated
- AssessmentPeriod is populated when applicable
- WhenAssessedGradeLevel is populated
- RetestIndicator logic is implemented and tested
- Event identity supports multiple attempts without collision

15.4 Safe Reprocessing

(Validates: Section 11.3, Validation 14.6)

Safe reprocessing must be tested across all required scenarios:

- Incremental reload
- Vendor correction reload
- Full-year reload
- Historical backfill

And must confirm:

- Duplicate prevention is enforced
- Stable natural keys are implemented
- Reloads do not generate duplicate StudentAssessment records
- Corrections update existing records rather than creating new ones

15.5 Scalability and Runtime

(Validates: Section 13.1 & 13.2, Validation 14.7)

- Batch scalability is validated under peak load conditions
- Retry logic is implemented and tested
- Backoff strategy is implemented and respected
- Parallelization is tested and safe across tenants
- Failure scenarios are tested
- Failures are logged clearly and are diagnosable

15.6 Governance and Namespace

(Validates: Sections 3, 10, 13.3, Validation 14.3 & 14.8)

- Vendor namespace usage is correctly implemented
- Default descriptor usage is validated
- Descriptor governance rules are followed
- AssessmentIdentifier stability is confirmed

- AssessmentFamily grouping is correctly implemented
- Version tracking is documented across:
    - Assessment
    - Ed-Fi Data Standard
    - Integration
- Descriptor mappings are:
    - Configurable per environment
    - Not hard-coded in transformation logic
    - Transparent and documented
    - Traceable over time
- Descriptor override capability exists and is externally configurable
- Integration behavior (identity resolution, descriptor mapping, event modeling) is explainable without inspecting transformation code